

Snapper Documentation

Comprehensive project guide

Generated on 2026-03-30

Table of Contents

[Snapper](#)

[API](#)

[Architecture](#)

[CLI](#)

[Configuration](#)

[Development](#)

[Messaging \(ZeroMQ\)](#)

[Trading Strategies](#)

Snapper

Trading platform with market data collection, trade runtime, and backtester. Supports Kraken, Zonda, Walutomat exchanges and Polygon.io data.

Quick Steps

```
# Initialize database with seed data and build static assets
make migrate-dev run-static

# Start the server
make run-server
```

Open <http://localhost:8000/> and log in:

- **Username:** admin
- **Password:** AdminSnapper2026!

Features

- **Market data collection** — WebSocket and REST API from Kraken, Zonda, Walutomat, Polygon.io
- **Trade runtime** — Facts-canonical live and paper trading with canonical Order and Execution facts, rebuildable Position and Balance projections, and optional durable command mode
- **Strategies** — Framework for creating strategies based on RSI, MACD, cointegration, and TA-Lib indicators
- **ZeroMQ messaging** — Pub/sub architecture for market data and signals
- **Provenance and audit** — Every payload item carries `session_id` and `sequence_id` for gap detection. Control table (always-on) and telemetry table (toggleable) record all commands and operational events
- **Web dashboard** — FastAPI + React with real-time WebSocket
- **CLI** — Full system management via Typer CLI
- **Backtesting** — Strategy testing on historical data

Quick Start

Requirements

- Python 3.14+
- Poetry
- Node.js 25+ and pnpm (for frontend)
- TA-Lib (C library)

Installation

```
# Clone repository
git clone https://github.com/mateusz-klatt/snapper.git
cd snapper

# Install system dependencies (macOS)
brew install ta-lib

# Install Python dependencies
make setup

# Install frontend dependencies
make ui-setup

# Copy and configure environment variables
cp .env.example .env
```

Configuration

Edit .env file:

```
# Database (SQLite dev, PostgreSQL prod)
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# Settings encryption in database
MASTER_PASSWORD=your_master_password

# HTTP Server
SERVER_HOST=127.0.0.1
SERVER_PORT=8000
SERVER_API_ONLY=false
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1

# Telemetry recording (pings, heartbeats, GET reads)
TELEMETRY_RECORDING_ENABLED=false

# ZeroMQ broker
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501
```

Running

```
# Initialize database and seed data
make migrate-dev

# Start server
snapper server
```

Dashboard available at <http://localhost:8000/>.

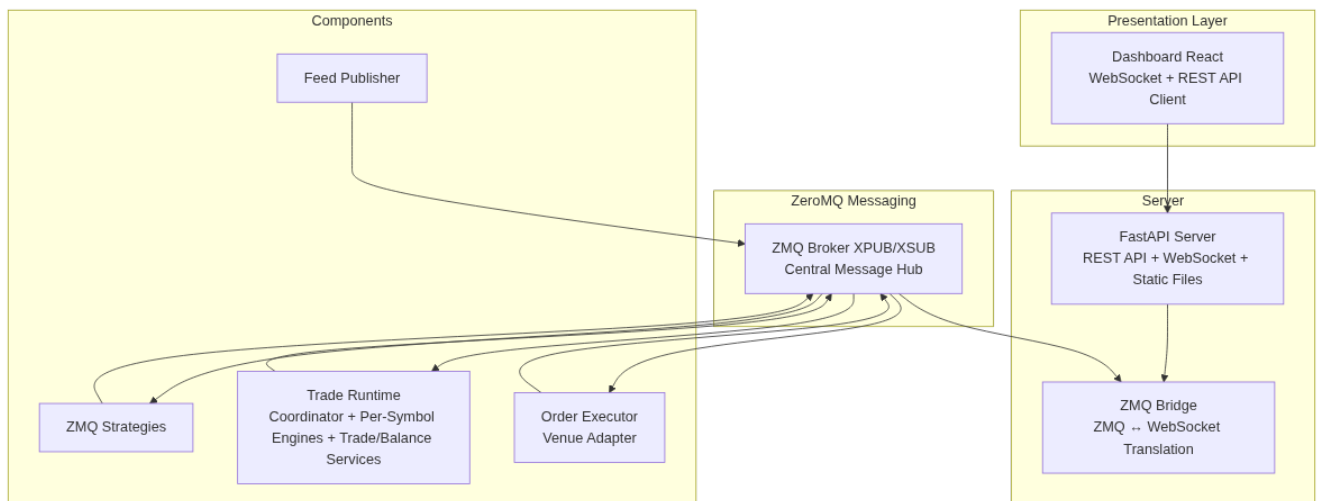
Trade Runtime Mode

The trade runtime supports two DB-backed modes controlled by the `use_durable_commands` setting:

- `false` (default) — dual-write mode. The engine writes durable command rows and still publishes directly to ZMQ.
- `true` — outbox-driven durable mode. Commands are published from the database outbox and executor venue-event persistence becomes fail-closed.

Restart snapper trade-zmq and the relevant executors after changing this setting.

System Overview



CLI Commands

Server and Infrastructure

```
snapper server          # Start FastAPI server
snapper broker          # Start ZMQ broker
snapper trade-zmq       # Trade runtime / coordinator
snapper executor        # Order executor
snapper feed            # Market data publisher
```

Database

```
snapper db-init         # Initialize schema
snapper db-upgrade      # Alembic migrations
snapper db-downgrade    # Rollback migrations
```

Users

```
snapper init-admin      # Create admin user
snapper list-users      # List users
snapper reset-password  # Reset password
```

Market Data

```
snapper update-kraken-symbols  # Sync Kraken symbols
snapper update-polygon-symbols # Sync Polygon symbols
snapper polygon-backfill-aggregates # Backfill historical data
snapper archive --day 2024-01-15 # Export candle cache to CSV
snapper archive --from 2024-01-01 --to 2024-01-31 --exchange polygon
snapper archive --table ticks --day 2024-01-15 --exchange polygon
snapper archive --table candles-audit --day 2024-01-15 --closed-only --purge
```

Strategies

Creating your own strategy:

```
from snapper.strategies.base import BaseStrategy, StrategySignal, StrategyConfig
from snapper.strategies.decorators import register_strategy,
create_strategy_process
from snapper.messaging.schemas.data import CandleData

@register_strategy("MyStrategy")
@create_strategy_process(
    process_name="my_strategy",
    default_config={
        "name": "my_strategy",
        "inputs": ["market.kraken.BTC-USD.candles.1h"],
        "outputs": ["BTC-USD"],
        "exchange": "paper",
```

```

        "params": {"threshold": 0.5},
    },
)
class MyStrategy(BaseStrategy):
    def __init__(self, config: StrategyConfig) -> None:
        super().__init__(config)
        self.threshold = self.params.get("threshold", 0.5)

    async def on_candle(self, instrument: str, candle: CandleData) ->
StrategySignal | None:
        if some_condition:
            return StrategySignal(
                instrument=instrument,
                side="buy",
                strength=1.0,
                price=candle.close,
                reason="Condition met",
            )
        return None

```

Development

Quality Gates

```

make check-all    # Full quality gate (backend + frontend + tests + 100%
coverage)
make fix-all       # Automatic formatting and linting fixes

```

Individual Steps

```

make fmt           # Check formatting
make lint          # Linting (ruff)
make typecheck     # Type checking (mypy)
make test          # Unit tests
make cov           # Tests with coverage (100% required)

```

Frontend

```

make ui-dev        # Vite development server
make ui-build      # Production build
make ui-lint       # ESLint
make ui-format     # Prettier

```

Docker

```
make docker-build-dev    # Build dev image
make docker-build-prod   # Build production image
make docker-run           # Run container
make docker-stop         # Stop container
```

Or with docker compose:

```
docker compose up -d
```

Documentation

Detailed documentation in [docs/](#) directory:

- [Architecture](#) — System structure and layers
- [Configuration](#) — Environment variables and settings
- [CLI](#) — Full command documentation
- [Strategies](#) — Creating trading strategies
- [API](#) — REST API and WebSocket
- [Messaging](#) — ZeroMQ architecture
- [Development](#) — Developer guidelines

License

MIT License — see [LICENSE](#) file.

API

Snapper provides a REST API and WebSocket interface for platform interaction. The API requires JWT authentication via HTTP-only cookies. Auth bootstrap endpoints under `/api/auth` are exempt, but other mutating requests (POST) require a valid X-CSRF-Token header.

Authentication

Authentication uses HTTP-only cookies. After login, the server sets `access_token`, `refresh_token`, and `csrf_token` cookies automatically.

Roles and Permissions

Role	Access
viewer	Read-only market data, orders, positions, strategies, system status
operator	Viewer permissions plus trade execution, process management
admin	Full access including user management and system configuration

POST /api/auth/login

Authenticate and create a session. Sets `access_token`, `refresh_token`, and `csrf_token` cookies on the response.

Request:

```
POST /api/auth/login
Content-Type: application/json

{
  "username": "admin",
  "password": "password123",
  "remember_me": false
}
```

Response (200):

```
{
  "message": "Login successful",
  "expires_in": 900,
  "user": {
    "username": "admin",
```

```
    "email": "admin@example.com",
    "role": "admin",
    "is_active": true,
    "created_at": "2026-01-10T08:00:00Z"
  }
}
```

Cookies set:

Cookie	HttpOnly	Secure	SameSite	Path	Max-Age	Description
access_token	Yes	Yes (prod)	Lax/Strict	/	session	JWT access token (15 min)
refresh_token	Yes	Yes (prod)	Lax/Strict	/api/auth	7 days	JWT refresh token
csrf_token	No	Yes (prod)	Lax/Strict	/	session	CSRF protection token

POST /api/auth/refresh

Refresh session tokens. The refresh_token cookie is sent automatically by the browser. Returns new tokens in cookies plus a WebSocket authentication token in the response body. This is the only way to obtain a ws_token.

Request:

```
POST /api/auth/refresh
```

Response (200):

```
{
  "message": "session refreshed",
  "ws_token": "eyJhbGciOi...",
  "ws_token_exp": "2026-01-18T12:30:00Z",
  "csrf_token": "abc123def456...",
  "user": {
    "username": "admin",
    "email": "admin@example.com",
    "role": "admin",
    "is_active": true,
    "created_at": "2026-01-10T08:00:00Z"
  }
}
```

POST /api/auth/logout

Logout and invalidate the current session. Clears all authentication cookies and blacklists tokens.

Request:

```
POST /api/auth/logout
```

Response (200):

```
{
  "message": "Logged out successfully"
}
```

GET /api/auth/me

Get the currently authenticated user's profile.

Request:

```
GET /api/auth/me
```

Response (200):

```
{
  "username": "admin",
  "email": "admin@example.com",
  "role": "admin",
  "is_active": true,
  "created_at": "2026-01-10T08:00:00Z"
}
```

CSRF Protection

The csrf_token cookie is readable by JavaScript (not HttpOnly). For authenticated mutating requests outside /api/auth/login, /api/auth/refresh, and /api/auth/logout, include its value as a header:

```
X-CSRF-Token: <value from csrf_token cookie>
```

New CSRF tokens are issued on login and refresh. There is no separate endpoint for obtaining CSRF tokens.

REST Endpoints

REST endpoints return the same Data schemas used by WebSocket messages (OrderData, SignalData, ExecutionData, PositionData, CandleData from `messaging.schemas.data`). This means the wire format is identical whether data arrives via REST or the WebSocket feed.

REST = Bulk WS

REST responses are JSON arrays of payload items. Each item carries its own per-item provenance (`public_id`, `session_id`, `sequence_id`) — there is no wrapper-level `public_id` and no custom HTTP headers for provenance. The shape of each array element is identical to what the WebSocket feed delivers for the same data type.

Provenance on Reads vs. Mutations

- **GET reads** — No client provenance is expected. The server records the request in the telemetry table for observability but does not stamp provenance onto the response items beyond what was stored at write time.
- **Mutations (POST)** — All mutations use command-style POST with a `PayloadRequest` envelope carrying provenance (`public_id`, `session_id`, `sequence_id`, `timestamp`) and domain intent in payload. The server-side `ClientProvenanceMiddleware` extracts these fields, emits a structured info log, and runs a per-session `GapDetector` to warn on sequence gaps. Gaps are logged as warnings but never reject requests.

ClientProvenanceMiddleware

`ClientProvenanceMiddleware` is an ASGI middleware that processes every mutation request:

1. Intercepts the request body transparently (replays chunks to downstream handlers).
2. Extracts `session_id`, `sequence_id`, and `public_id` from the JSON body when present.
3. Emits a structured log with `client_session_id`, `client_sequence_id`, `client_public_id`, and the request path.
4. Runs a per-session `GapDetector` to detect sequence gaps from the client.
5. Records the mutation in the control table via a finally block, so the row is persisted whether the request succeeded or failed.

The control recording is non-blocking: any DB failure is logged and swallowed so the response already sent to the client is never invalidated. The control row includes the redacted request payload, HTTP method, path, outcome (ok, error, or exception), and server-side provenance (`session_id` and `sequence_id` from the middleware's own `SequenceTracker`).

All data endpoints support bitemporal querying via the optional `as_of` parameter (UTC datetime). When provided, the query returns data as it was known at that point in time. When omitted, the current time is used.

GET /api/health

Public health check endpoint. No authentication required.

status reflects the health of enabled long-running CORE processes: "healthy" when all are running, "error" if any are missing. In SERVER_API_ONLY mode the status is always "healthy" (no processes are started by design).

Response (200):

```
{
  "type": "health_check",
  "status": "healthy",
  "timestamp": "2026-01-18T12:00:00Z",
  "version": "0.1.0",
  "connections": {
    "type": "connection_stats",
    "active_connections": 5,
    "zmq_subscribers": 12,
    "subscriber_tasks": 12,
    "active_topics": 8,
    "active_clients": 3
  },
  "topics": {
    "type": "health_topics",
    "available": 7,
    "active": 3
  },
  "gap_detection": {
    "type": "gap_detection_stats",
    "bridge": {
      "type": "gap_stats",
      "gaps_detected": 0,
      "session_resets": 0,
      "duplicates": 0,
      "mid_stream_joins": 0,
      "rejected_unstamped": 0
    },
    "rest_clients": {}
  }
}
```

The `gap_detection` field provides observability into sequence gap detection across the ZMQ bridge and per-session REST client detectors.

GET /api/candles

Fetch OHLCV candlestick data. Requires `read:market_data` permission.

Request:

```
GET /api/candles?instrument=BTC-USD&exchange=kraken&timeframe=1h&limit=100
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
instrument	string	yes	Instrument symbol (e.g., BTC-USD)
exchange	string	yes	Exchange name (kraken, zonda, walumat, polygon)
timeframe	string	yes	Candle timeframe (e.g., 1m, 5m, 15m, 1h, 4h, 1d)
limit	int	no	Number of candles, max 1000 (default 100)
as_of	datetime	no	Point-in-time query, UTC (default: current time)

Returns 204 No Content if the instrument is not found.

Response (200):

```
[
  {
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "candle",
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "timeframe": "1h",
    "open_at": "2026-01-18T11:00:00Z",
    "open": 42000.0,
    "high": 42500.0,
    "low": 41800.0,
    "close": 42300.0,
    "volume": 1234.56,
    "vwap": 42150.0,
    "trades": 5678
  }
]
```

GET /api/orders

Fetch orders with optional filtering. Requires read:orders permission.

Request:

```
GET /api/orders?symbol=BTC-USD&exchange=kraken&limit=100&offset=0
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
symbol	string	no	Filter by instrument symbol
exchange	string	no	Filter by exchange (paper, kraken, zonda, walutomat)
limit	int	no	Number of orders, 1-1000 (default 100)
offset	int	no	Number of orders to skip (default 0)
as_of	datetime	no	Point-in-time query, UTC (default: current time)

Response (200):

```
[
  {
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "order",
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "client_order_id": "signal-a1b2c3d4",
    "exchange_order_id": "KRAKEN-456",
    "created_at": "2026-01-18T12:00:00Z",
    "updated_at": "2026-01-18T12:01:00Z",
    "side": "buy",
    "order_type": "limit",
    "price": 42000.0,
    "size": 0.1,
    "filled_size": 0.1,
    "average_price": 42000.0,
    "status": "filled",
    "time_in_force": "GTC",
    "mode": "live",
    "error": null
  }
]
```

GET /api/signals

Fetch trading signals with optional filtering. Requires read:market_data permission.

Request:

```
GET /api/signals?instrument=BTC-
USD&strategy=rsi_btc_1h&exchange=paper&hours=24&limit=100
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
instrument	string	no	Filter by instrument symbol

Parameter	Type	Required	Description
strategy	string	no	Filter by strategy name
exchange	string	no	Filter by exchange (paper, kraken, zonda, walutomat)
hours	int	no	Hours of history, max 168 (default 24)
limit	int	no	Number of signals, max 1000 (default 100)
as_of	datetime	no	Point-in-time query, UTC (default: current time)

Response (200):

```
[
  {
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "signal",
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "paper",
    "side": "buy",
    "strength": 0.85,
    "reason": "RSI 28.5 <= 30",
    "strategy_name": "rsi_btc_1h",
    "price": 42000.0,
    "fired_at": "2026-01-18T12:00:00Z"
  }
]
```

GET /api/executions

Fetch order fills/executions. Requires read:orders permission.

Request:

```
GET /api/executions?limit=100
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
limit	int	no	Number of executions, max 1000 (default 100)
as_of	datetime	no	Point-in-time query, UTC (default: current time)

Response (200):

```
[
  {
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
```

```
    "type": "execution",
    "timestamp": "2026-01-18T12:01:00Z",
    "trade_id": "TTRAD-456",
    "exchange_order_id": "KRAKEN-456",
    "client_order_id": "signal-a1b2c3d4",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "side": "buy",
    "size": 0.1,
    "price": 42000.0,
    "fee": 0.001,
    "fee_asset": "USD",
    "status": "filled",
    "executed_at": "2026-01-18T12:00:59Z"
  }
]
```

GET /api/positions

Fetch current portfolio positions. Requires read:positions permission.

Request:

```
GET /api/positions
X-CSRF-Token: <csrf_token>
```

Parameters:

Parameter	Type	Required	Description
as_of	datetime	no	Point-in-time query, UTC (default: current time)

Response (200):

```
[
  {
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "position",
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "quantity": 0.5,
    "average_price": 41500.0,
    "unrealized_pnl": 250.0,
    "realized_pnl": 100.0,
    "mode": "live"
  }
]
```

GET /api/exchanges

List distinct exchange names from active symbol aliases. Requires read:market_data permission.

Request:

```
GET /api/exchanges
X-CSRF-Token: <csrf_token>
```

Response (200):

```
["kraken", "polygon", "walutomat", "zonda"]
```

GET /api/exchanges/{exchange}/instruments

List distinct native symbols available on a given exchange. Requires read:market_data permission.

Request:

```
GET /api/exchanges/kraken/instruments
X-CSRF-Token: <csrf_token>
```

Response (200):

```
["BTC-USD", "ETH-USD", "SOL-USD"]
```

GET /api/status

System-wide status including trader process, backtests, and active strategies. Requires read:system_status permission.

Request:

```
GET /api/status
X-CSRF-Token: <csrf_token>
```

Response (200):

```
{
  "trader": {
    "status": "running",
    "pid": null,
    "started_at": null,
    "command": null,
    "exit_code": null,
    "error": null
  }
}
```

```

},
"backtests": {},
"strategies": [
  {
    "strategy_name": "rsi_btc_1h",
    "status": "running",
    "details": {},
    "signals_generated": 42,
    "trades_executed": 5,
    "last_signal": "buy",
    "last_signal_time": "2026-01-18T11:45:00Z",
    "pnl": 150.25,
    "pid": 12345,
    "uptime": "2h 15m"
  }
]
}

```

GET /api/ws/stats

WebSocket and ZMQ bridge statistics. Requires read:system_status permission.

Request:

```

GET /api/ws/stats
X-CSRF-Token: <csrf_token>

```

Response (200):

```

{
  "websocket": {
    "active_connections": 5,
    "topic_subscribers": {
      "market.kraken.BTC-USD.candles.1h": 3,
      "signals.paper.BTC-USD.rsi_btc_1h": 2
    },
    "client_count": 5
  },
  "zmq_bridge": {
    "active_topics": 12,
    "subscriber_tasks": 12,
    "available_topics": ["admin", "market", "orders.commands",
"orders.events", "signals", "strategy.signals", "system.heartbeats."],
  },
  "connections": {
    "active_connections": 5,
    "zmq_subscribers": 12,
    "subscriber_tasks": 12,
    "active_topics": 8,
    "active_clients": 5
  },
  "topics": {
    "market.kraken.BTC-USD.candles.1h": {
      "active_subscribers": 3,
      "received": 1200,
      "forwarded": 1180,

```

```

        "throttled": 20,
        "dropped": 0,
        "timeout": 0,
        "errors": 0,
        "invalid_messages": 0,
        "last_message_ts": 1737208800.0,
        "throttle_ms": 100,
        "pattern": "market."
    },
    },
    "subscriptions": {
        "per_topic": {
            "market.kraken.BTC-USD.candles.1h": 3
        },
        "per_client": {
            "140234567890": ["market.kraken.BTC-USD.candles.1h"]
        }
    },
    "config": {
        "broker_xpub": "tcp://127.0.0.1:7501",
        "heartbeat_interval_ms": 15000
    }
}

```

GET /api/zmq/health

ZMQ bridge health check. Requires read:system_status permission.

Request:

```

GET /api/zmq/health
X-CSRF-Token: <csrf_token>

```

Response (200):

```

{
  "status": "healthy",
  "timestamp": "2026-01-18T12:00:00Z",
  "components": {
    "zmq_context": "ok",
    "websocket_manager": "ok",
    "active_connections": 5
  },
  "config": {
    "available_topics": ["admin", "market", "orders.commands",
"orders.events", "signals", "strategy.signals", "system.heartbeats."],
  },
  "connections": {
    "active_connections": 5,
    "zmq_subscribers": 12,
    "subscriber_tasks": 12,
    "active_topics": 8,
    "active_clients": 3
  },
}

```

```
"message_stats": {},
"errors": []
}
```

Process Management

Process endpoints manage background services (feeds, strategies, executors, brokers). Most require `manage:processes` permission (operator/admin). The summary endpoint requires only `read:system_status` (viewer+).

GET /api/processes/available

List registered process templates that can be instantiated. Requires `manage:processes` permission.

Response (200):

```
{
  "processes": [
    {
      "name": "zmq_broker",
      "class_path":
"snapper.messaging.infrastructure.broker.ZmqBrokerProcess",
      "method": "start",
      "description": "ZeroMQ XPUB/XSUB message broker",
      "lifecycle": "long_running",
      "role": "core",
      "tags": ["infrastructure"],
      "parameters_schema": null
    }
  ],
  "count": 1
}
```

GET /api/processes/configured

List configured process instances with runtime state. Requires `manage:processes` permission.

Response (200):

```
{
  "processes": [
    {
      "name": "zmq_broker",
      "enabled": true,
      "running": true,
      "mode": "thread",
      "class_path":
"snapper.messaging.infrastructure.broker.ZmqBrokerProcess",

```

```

        "method": "start",
        "parameters": {},
        "note": null,
        "lifecycle": "long_running",
        "role": "core",
        "tags": ["infrastructure"],
        "parameters_schema": null,
        "is_one_shot": false,
        "active_public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b"
    },
    "count": 1
}

```

GET /api/processes/summary

Lightweight process category counts for the overview dashboard. Requires read:system_status permission.

Response (200):

```

{
  "feeds": { "running": 2, "total": 3 },
  "strategies": { "running": 1, "total": 2 },
  "executors": { "running": 1, "total": 1 },
  "brokers": { "running": 1, "total": 1 }
}

```

POST /api/processes

Create a new process configuration from a registered template. Requires manage:processes permission. Returns 201 on success.

Request:

```

POST /api/processes
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
  "name": "kraken_feed_btc",
  "template": "kraken_feed_publisher",
  "enabled": true,
  "mode": "thread",
  "parameters": { "symbols": ["BTC-USD"] },
  "note": "Kraken BTC feed"
}

```

Response (201):

```
{
  "status": "created",
  "process": {
    "name": "kraken_feed_btc",
    "template": "kraken_feed_publisher"
  }
}
```

GET /api/processes/schema/{name}

Get the configuration schema and defaults for a registered process template. Requires manage:processes permission.

Response (200):

```
{
  "name": "kraken_feed_publisher",
  "description": "Kraken market data feed publisher",
  "class_path":
    "snapper.messaging.publishers.kraken.KrakenMarketDataPublisher",
  "method": "start",
  "default_enabled": true,
  "default_mode": "thread",
  "default_parameters": {},
  "lifecycle": "long_running"
}
```

POST /api/processes/{name}/start

Start a configured process. Requires manage:processes permission.

Request:

```
POST /api/processes/zmq_broker/start
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
  "mode": "process"
}
```

All fields are optional. mode overrides the stored execution mode for this run only — it does not persist to Settings. Omitting it uses the stored value.

Response (200):

```
{
  "status": "success",
  "name": "zmq_broker",
}
```

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "message": "Process started"
}
```

POST /api/processes/{name}/stop

Stop a running process. Requires manage:processes permission.

Request:

```
POST /api/processes/zmq_broker/stop
X-CSRF-Token: <csrf_token>
```

Response (200):

```
{
  "status": "success",
  "name": "zmq_broker",
  "message": "Process stopped"
}
```

GET /api/processes/runs

List historical process runs. Requires manage:processes permission.

Parameters:

Parameter	Type	Required	Description
limit	int	no	Number of runs to return (default 50)
name	string	no	Filter by process name

Response (200):

```
{
  "runs": [
    {
      "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
      "process_name": "zmq_broker",
      "status": "succeeded",
      "role": "core",
      "lifecycle": "long_running",
      "parameters": {},
      "result": null,
      "error": null,
      "tags": ["infrastructure"],
      "started_at": "2026-01-18T10:00:00Z",
      "completed_at": "2026-01-18T18:00:00Z"
    }
  ]
}
```

```
  ],
  "count": 1
}
```

Strategies

GET /api/strategies

List configured strategy processes with lightweight status. Requires read:strategies permission (viewer+).

Response (200):

```
{
  "strategies": [
    {
      "name": "strategy_rsi_btc_1h",
      "running": true,
      "enabled": true,
      "mode": "process"
    }
  ],
  "count": 1
}
```

Settings

Settings endpoints require configure:system permission (admin only).

GET /api/settings

List all settings, optionally filtered by category.

Parameters:

Parameter	Type	Required	Description
category	string	no	Filter by setting category

Response (200):

```
[
  {
    "key": "kraken_api_key",
    "value": "xK9m...",
    "category": "exchanges",
    "description": "Kraken REST API key",
    "updated_at": "2026-01-15T10:00:00Z",
  }
]
```

```
    "updated_by": "admin"
  }
}
```

GET /api/settings/categories

List distinct setting category names.

Response (200):

```
{
  "categories": ["exchanges", "strategy", "system"]
}
```

POST /api/settings/{key}/set

Set (create or update) a setting value.

Request:

```
POST /api/settings/kraken_api_key/set
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
  "public_id": "<uuid7>",
  "session_id": "<client-session>",
  "sequence_id": 1,
  "timestamp": "2026-01-18T12:00:00Z",
  "payload": {
    "value": "new-api-key-value",
    "category": "exchanges",
    "description": "Kraken REST API key"
  }
}
```

Payload fields:

Field	Type	Required	Description
value	string	yes	Setting value
category	string	no	Setting category (default: system)
description	string	no	Human-readable description

Response (200):

```
{
  "type": "setting_response",
  "public_id": "<uuid7>",
  "session_id": "<server-session>",
```

```

    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
      "type": "setting_read",
      "key": "kraken_api_key",
      "value": "new-api-key-value",
      "category": "exchanges",
      "description": "Kraken REST API key",
      "updated_at": "2026-01-18T12:00:00Z",
      "updated_by": "admin"
    }
  }
}

```

POST /api/settings/{key}/remove

Soft-delete a setting by key (sets known_to to current time).

Request:

```

POST /api/settings/kraken_api_key/remove
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
  "public_id": "<uuid7>",
  "session_id": "<client-session>",
  "sequence_id": 2,
  "timestamp": "2026-01-18T12:00:01Z",
  "payload": {}
}

```

Response (200):

```

{
  "type": "message",
  "public_id": "<uuid7>",
  "session_id": "<server-session>",
  "sequence_id": 2,
  "timestamp": "2026-01-18T12:00:01Z",
  "payload": "Setting 'kraken_api_key' deleted successfully"
}

```

WebSocket

Connection and Authentication

The WebSocket endpoint is at /api/ws. Authentication is message-based, not query-parameter-based.

Connection flow:

1. Client connects to ws://host:port/api/ws (or wss:// over TLS)
2. Server accepts the connection and validates the origin header
3. Server sends auth_required message
4. Client sends authenticate message with a WebSocket token
5. Server validates the token and sends auth_ok
6. Server sends auth_complete with available topics for the user's role
7. Client can now subscribe to topics

Obtaining a WebSocket token:

Call POST /api/auth/refresh (requires a valid refresh_token cookie). The response includes ws_token and ws_token_exp fields.

Authentication Messages

Server sends after connection:

```
{
  "type": "auth_required",
  "timeout": 10,
  "timestamp": "2026-01-18T12:00:00Z"
}
```

Client sends to authenticate:

```
{
  "type": "authenticate",
  "ws_token": "eyJhbGciOi...",
  "timestamp": "2026-01-18T12:00:00Z"
}
```

Server sends on success:

```
{
  "type": "auth_ok",
  "exp": "2026-01-18T12:30:00Z",
  "timestamp": "2026-01-18T12:00:00Z"
}
```

Server sends session info:

```
{
  "type": "auth_complete",
  "available_topics": [
    "market",
    "orders.commands",
    "orders.events",
    "signals",
  ]
}
```

```
    "strategy.signals",
    "system.heartbeats."
  ],
  "user_role": "operator",
  "session_expires_at": "2026-01-18T12:15:00Z",
  "ws_token_exp": "2026-01-18T12:30:00Z",
  "timestamp": "2026-01-18T12:00:00Z"
}
```

available_topics contains allowed topic roots/pattern keys for the user's role. Concrete subscriptions may use those roots as prefixes or full topic strings such as market.kraken.BTC-USD.candles.1h or signals.paper.BTC-USD.rsi_btc_1h.

Server sends on failure:

```
{
  "type": "auth_failed",
  "reason": "Invalid token",
  "timestamp": "2026-01-18T12:00:00Z"
}
```

Reauthentication

Before the WebSocket token expires, the server sends a reauth_required message. Clients may also refresh proactively before that deadline. In both cases they obtain a new ws_token via POST /api/auth/refresh and send it as a reauth message.

Server warning:

```
{
  "type": "reauth_required",
  "deadline": "2026-01-18T12:29:00Z",
  "timestamp": "2026-01-18T12:28:00Z"
}
```

Client sends new token:

```
{
  "type": "reauth",
  "ws_token": "eyJhbGciOiI... (new token) ...",
  "timestamp": "2026-01-18T12:28:30Z"
}
```

Server confirms:

```
{
  "type": "reauth_ok",
  "exp": "2026-01-18T13:00:00Z",
  "timestamp": "2026-01-18T12:28:30Z"
}
```

If the client does not reauthenticate in time:

```
{
  "type": "auth_expired",
  "timestamp": "2026-01-18T12:30:00Z"
}
```

Subscription Management

Subscribe to topics:

```
{
  "type": "subscribe",
  "topics": ["market.kraken.BTC-USD.candles.1h", "signals.paper.BTC-USD.rsi_btc_1h"]
}
```

Server confirms subscription:

```
{
  "type": "subscription_success",
  "action": "subscribe",
  "status": "subscribed",
  "topics": ["market.kraken.BTC-USD.candles.1h", "signals.paper.BTC-USD.rsi_btc_1h"],
  "denied_topics": [],
  "active_subscriptions": ["market.kraken.BTC-USD.candles.1h", "signals.paper.BTC-USD.rsi_btc_1h"],
  "message": null,
  "timestamp": "2026-01-18T12:00:01Z"
}
```

Unsubscribe:

```
{
  "type": "unsubscribe",
  "topics": ["market.kraken.BTC-USD.candles.1h"]
}
```

List active subscriptions:

```
{
  "type": "get_subscriptions"
}
```

Response:

```
{
  "type": "subscriptions_list",
  "subscriptions": ["signals.paper.BTC-USD.rsi_btc_1h"],
  "available_topics": ["market", "orders.commands", "orders.events", "signals", "strategy.signals", "system.heartbeats."],
}
```

```
{
  "total_available": 6,
  "timestamp": "2026-01-18T12:00:02Z"
}
```

Ping/Pong

Client sends:

```
{
  "type": "ping"
}
```

Server responds:

```
{
  "type": "pong",
  "timestamp": "2026-01-18T12:00:00Z",
  "active_connections": 5
}
```

Server Data Messages

Data messages are flat JSON objects forwarded directly from the ZMQ bus. There is no "data" wrapper. Messages with malformed JSON are dropped by the bridge before forwarding and are counted in the `invalid_messages` metric for the topic.

All data messages carry `session_id` and `sequence_id` provenance fields (see REST section above). All WebSocket control messages (auth, subscribe, ping/pong, errors) inherit from `StrictDataSchema`, so they also carry the same provenance envelope. Clients can use these fields to detect gaps without server-side replay support.

Candle

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "candle",
  "timestamp": "2026-01-18T11:00:00Z",
  "session_id": "019e0000-0000-7000-0000-000000000001",
  "sequence_id": 42,
  "instrument": "BTC-USD",
  "exchange": "kraken",
  "timeframe": "1h",
  "open_at": "2026-01-18T11:00:00Z",
  "open": 42000.0,
  "high": 42500.0,
  "low": 41800.0,
  "close": 42300.0,
  "volume": 1234.56,
}
```

```
"vwap": 42150.0,
"trades": 5678
}
```

Tick

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "tick",
  "timestamp": "2026-01-18T12:00:00Z",
  "instrument": "BTC-USD",
  "exchange": "kraken",
  "volume": 1234.5,
  "bid": 42000.0,
  "ask": 42001.0,
  "last": 42000.5
}
```

Trade

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "trade",
  "timestamp": "2026-01-18T12:00:00Z",
  "instrument": "BTC-USD",
  "exchange": "kraken",
  "executed_at": "2026-01-18T12:00:00Z",
  "price": 42000.5,
  "volume": 0.25,
  "side": "buy"
}
```

Signal

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "signal",
  "timestamp": "2026-01-18T12:00:00Z",
  "session_id": "019e0000-0000-7000-0000-000000000001",
  "sequence_id": 7,
  "instrument": "BTC-USD",
  "exchange": "paper",
  "side": "buy",
  "strength": 0.85,
  "reason": "RSI 28.5 <= 30",
  "price": 42000.0,
  "strategy_name": "rsi_btc_1h",
  "fired_at": "2026-01-18T12:00:00Z"
}
```

Execution

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "execution",
  "timestamp": "2026-01-18T12:01:00Z",
  "trade_id": "TTRAD-456",
  "exchange_order_id": "KRAKEN-456",
  "client_order_id": "signal-a1b2c3d4",
  "instrument": "BTC-USD",
  "exchange": "kraken",
  "side": "buy",
  "size": 0.1,
  "price": 42000.0,
  "fee": 0.001,
  "fee_asset": "USD",
  "status": "filled",
  "executed_at": "2026-01-18T12:00:59Z"
}
```

Order

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "order",
  "timestamp": "2026-01-18T12:00:00Z",
  "exchange_order_id": "KRAKEN-456",
  "client_order_id": "signal-a1b2c3d4",
  "instrument": "BTC-USD",
  "exchange": "kraken",
  "side": "buy",
  "status": "submitted",
  "order_type": "limit",
  "size": 0.1,
  "filled_size": 0.0,
  "price": 42000.0,
  "average_price": null,
  "reason": null,
  "time_in_force": "GTC",
  "error": null,
  "created_at": "2026-01-18T12:00:00Z",
  "updated_at": null
}
```

Heartbeat

```
{
  "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
  "type": "heartbeat",
  "timestamp": "2026-01-18T12:00:00Z",
  "session_id": "019e1a2b-0000-7000-8000-000000000001",
  "sequence_id": 42,
  "component": "zmq_broker",
  "sequence": 42,
  "status": "healthy",
}
```

```
"lag_ms": 5,  
"meta": {}  
}
```

Error

```
{  
  "type": "error",  
  "message": "Topic 'invalid.topic' does not exist",  
  "timestamp": "2026-01-18T12:00:00Z"  
}
```

Available Topic Patterns

Topics use dot-separated hierarchical names. Subscribe using the full topic string or a prefix to match multiple topics.

Market Data

- market.{exchange}.{instrument}.candles.{timeframe} -- OHLCV candles
- market.{exchange}.{instrument}.ticks -- Price ticks
- market.{exchange}.{instrument}.trades -- Individual trades
- market.paper.{source_exchange}.{instrument}.candles.{timeframe} -- Paper candles sourced from a real exchange
- market.paper.{source_exchange}.{instrument}.ticks -- Paper ticks sourced from a real exchange

Signals

- signals.{exchange}.{instrument}.live -- Live trading signals
- signals.paper.{instrument}.{strategy_name} -- Paper trading signals

Order Events

- orders.events.{exchange}.{instrument}.submitted -- Order submitted
- orders.events.{exchange}.{instrument}.accepted -- Order accepted
- orders.events.{exchange}.{instrument}.rejected -- Order rejected
- orders.events.{exchange}.{instrument}.executed -- Order fill
- orders.events.{exchange}.{instrument}.cancelled -- Order cancelled
- orders.events.{exchange}.{instrument}.expired -- Order expired
- orders.events.{exchange}.{instrument}.replaced -- Order replaced

Order Commands

- `orders.commands.{exchange}.{instrument}.submit -- Submit order`
- `orders.commands.{exchange}.{instrument}.cancel -- Cancel order`
- `orders.commands.{exchange}.{instrument}.replace -- Replace order`

System

- `system.heartbeats.{component}.{name}[.{source}] -- Component heartbeats`
- `admin.{resource} -- Administrative events (admin only)`

Error Handling

HTTP Status	Description
401	Missing or invalid authentication
403	Insufficient permissions or invalid CSRF token
404	Resource not found
422	Request validation error
429	Rate limit exceeded (Retry-After header included)
500	Internal server error

Usage Example (Python)

```
import httpx

BASE_URL = "http://localhost:8000/api"

async def main():
    async with httpx.AsyncClient() as client:
        login_resp = await client.post(
            f"{BASE_URL}/auth/login",
            json={"username": "admin", "password": "password123"},
        )
        csrf_token = client.cookies.get("csrf_token")

        refresh_resp = await client.post(f"{BASE_URL}/auth/refresh")
        ws_token = refresh_resp.json()["ws_token"]

        candles_resp = await client.get(
            f"{BASE_URL}/candles",
            params={
                "instrument": "BTC-USD",
                "exchange": "kraken",
                "timeframe": "1h",
            },
        ),
```

```
        headers={"X-CSRF-Token": csrf_token},
    )
    candles = candles_resp.json()
    print(candles)
```

Usage Example (JavaScript)

This example is intentionally minimal. The shipped frontend client reuses cached WS tickets, schedules proactive reauthentication before expiry, and sends heartbeat pings every 5 seconds.

```
async function connect() {
    const refreshResp = await fetch("/api/auth/refresh", {
        method: "POST",
        credentials: "include",
    });
    const { ws_token } = await refreshResp.json();

    const protocol = location.protocol === "https:" ? "wss:" : "ws:";
    const ws = new WebSocket(`${protocol}://${location.host}/api/ws`);

    ws.onopen = () => {
        console.log("Connected, waiting for auth_required...");
    };

    ws.onmessage = (event) => {
        const msg = JSON.parse(event.data);

        if (msg.type === "auth_required") {
            ws.send(JSON.stringify({
                type: "authenticate",
                ws_token: ws_token,
            }));
        }

        if (msg.type === "auth_complete") {
            console.log("Authenticated. Topics:", msg.available_topics);
            ws.send(JSON.stringify({
                type: "subscribe",
                topics: ["market.kraken.BTC-USD.candles.1h"],
            }));
        }

        if (msg.type === "candle" || msg.type === "tick") {
            console.log("Data:", msg);
        }

        if (msg.type === "reauth_required") {
            refreshAndReauth(ws);
        }
    };

    setInterval(() => {
        if (ws.readyState === WebSocket.OPEN) {
            ws.send(JSON.stringify({ type: "ping" }));
        }
    }, 5000);
}
```

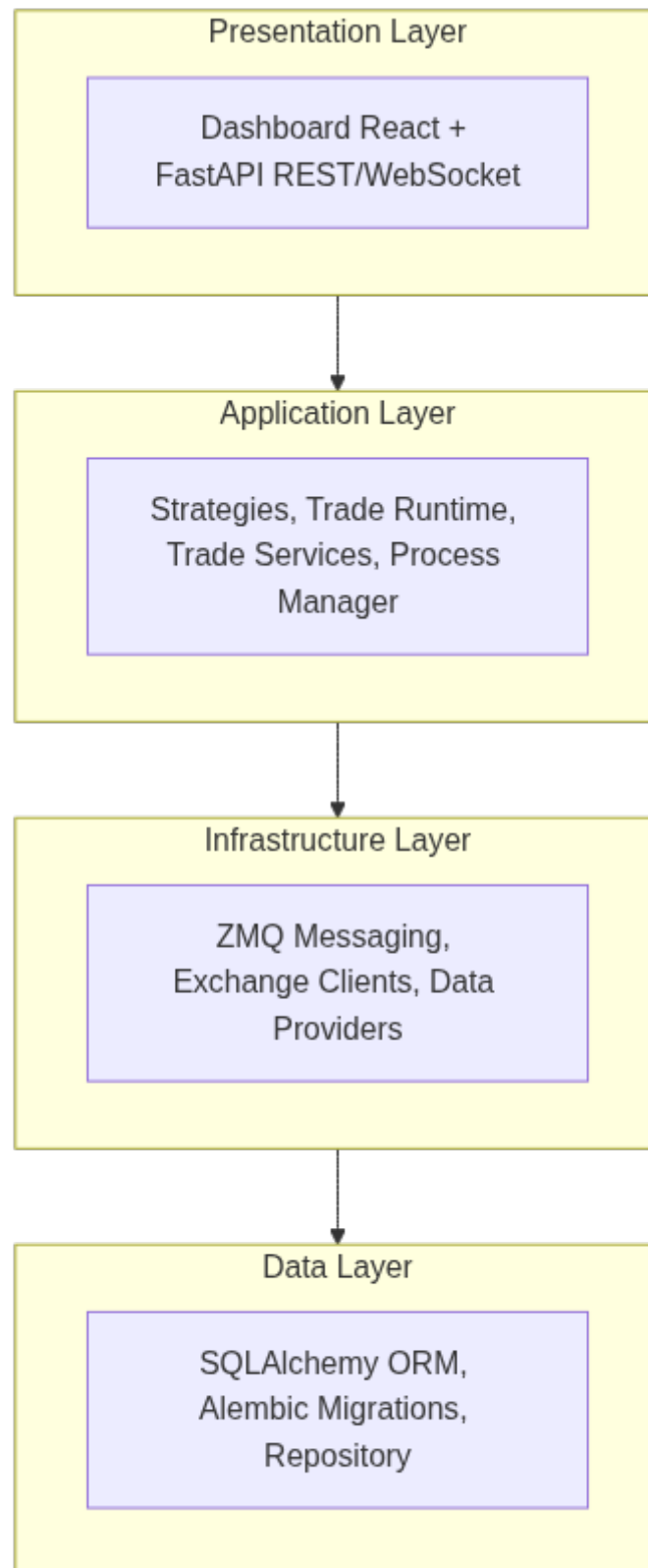
```
    }  
    }, 30000);  
}
```

Architecture

Snapper is a trading platform built with a layered architecture using asynchronous processing and ZeroMQ messaging.

The trading path is facts-canonical: Order and Execution are the canonical business facts, while Position, Balance, and equity are derived projections that can be rebuilt from facts plus checkpoints.

System Layers



Components

Core (src/snapper/core/)

Fundamental types and aliases used throughout the application:

- TradeSide — Trade direction (buy, sell)
- OrderType — Order type (market, limit, stop, stop_limit)
- OrderStatus — Order status in lifecycle
- ExecutionMode — Execution mode (live, paper)
- OrderExchange — Order-capable exchanges (paper, kraken, zonda, walutomat)

Config (src/snapper/config/)

Two-layer configuration:

1. Bootstrap Settings (bootstrap.py)

Settings from environment variables required before database connection:

- Database URL
- Master password for encryption
- HTTP and ZMQ server endpoints

2. App Settings (app.py)

Settings from database (encrypted):

- Exchange API keys
- Trading parameters
- Authentication configuration

Data (src/snapper/data/)

Persistence layer with SQLAlchemy:

- **ORM Models** (models.py):
 - Instrument — Financial instruments (natural key: symbol_public_id + exchange). Instrument.public_id is the stable identity used by fact tables (orders, executions, positions, signals, candles). The versioned business attributes symbol_public_id and exchange are resolved via ensure_instrument()
 - Candle — OHLCV candles
 - Trade — Transactions
 - Order — Canonical order lifecycle facts (includes mode: "live" or "paper")
 - Execution — Canonical confirmed fills
 - Position — Derived position projection (includes mode: "live" or "paper")
 - TradeCommand — Durable trade intent written by the engine before execution

- VenueEvent — Durable venue observations and acknowledgements persisted by executors
- TradeProjectionCheckpoint — Materialized position/balance snapshot for fast recovery
- Signal — Signal events
- User — System users
- Setting — Settings (encrypted)
- Symbol — Symbol identity with versioned attributes (native_symbol, base, quote, asset_type) via SCD Type 2. Archive symbols for filesystem paths are derived from the anchor row (first version per public_id)
- SymbolAlias — Exchange-specific symbol aliases (one row per native/exchange/channel)
- SymbolExchangeCapability — Exchange-specific symbol capabilities
- ProcessRun — Background process execution records
- InstrumentSpec — Instrument trading specifications
- MarketSnapshot — Real-time market data snapshots
- UserLoginEvent — Authentication event log
- Control — Always-on audit for commands, auth events, subscribe/unsubscribe, REST mutations. Includes redacted payload, outcome, and client causation linkage (client_session_id, client_public_id)
- Telemetry — Toggleable high-volume table for pings, heartbeats, pongs, and GET read requests. Recording gated by TELEMETRY_RECORDING_ENABLED setting

All ORM models carry provenance via TemporalMixin:

- session_id (str, required) — producer session identity
- sequence_id (int, required) — per-table monotonic counter for gap detection

StrictDataSchema (Pydantic base) requires these fields at construction. Producers must obtain values from a SequenceTracker before creating the event, ensuring every payload is complete and identifiable from birth.

All ORM models use a dual-key identity pattern:

- id (INTEGER, internal PK) — row/version identifier for SCD2 close operations: SELECT → lock → close by id → INSERT new version. Never exposed outside the repository layer.
- public_id (UUID7 string) — logical entity identifier for domain joins and external references. Stable across SCD2 versions.
- timestamp (DateTime, system time / known_from)
- known_to (DateTime NOT NULL, default KNOWN_TO_MAX = 9999-12-31T23:59:59 UTC, active rows have known_to == KNOWN_TO_MAX; query pattern: WHERE timestamp <= :t AND known_to > :t)

No hard foreign keys between temporal entities. All domain joins use `child.instrument_public_id == Instrument.public_id` (or `Execution.order_public_id == Order.public_id`) with temporal filters. This enables future archival of closed SCD2 rows without breaking referential integrity.

- **SQLAlchemyRepository** (repository.py) — Async CRUD for SQLite/PostgreSQL
- **DatabaseRepository** (repository.py) — Sync access for scripts, archiver, and background updaters
- **CandleCacheArchiver** (archiver.py) — Exports active candle data to polygon-compatible CSV cache files per exchange/archive_symbol
- **EventArchiver** (archiver.py) — Exports append-only event rows (Tick, Trade, Signal, Execution, Telemetry, Control) to per-day CSV archive files with full temporal metadata, merge/dedup, and optional purge
- **CandleAuditArchiver** (archiver.py) — Exports all candle SCD2 versions (closed + active) with full temporal metadata, grouped by open_at date
- **StateArchiver** (archiver.py) — Generic archiver for state-SCD2 tables (Order, Position, Instrument, Setting, Symbol, etc.) with closed_only filtering and Symbol anchor row protection
- **Archive symbol resolution** (archive_symbols.py) — Stable filesystem-safe symbol naming derived from Symbol anchor rows, with seniority-based collision handling

Strategies (src/snapper/strategies/)

Trading strategies framework:

- **BaseStrategy** (base.py) — Base class for all strategies
- **StrategySignal** — Trading signal structure
- **StrategyConfig** — Strategy configuration

Built-in strategies:

- **RSIReversion** (rsi.py) — Mean reversion on RSI
- **MACDCrossover** (macd.py) — MACD crossover
- **CointegrationPairs** (cointegration.py) — Pairs trading

Indicators (src/snapper/indicators/)

Technical indicators:

- **rsi** — Relative Strength Index (Python implementation)
- **macd** — Moving Average Convergence Divergence
- **ta_lib_adapter** — TA-Lib library adapter

Messaging (src/snapper/messaging/)

ZeroMQ pub/sub architecture:



Components:

- **Broker** (infrastructure/broker.py) — XPUB/XSUB proxy
- **Publishers** (publishers/) — Market data publication
- **Executors** (executors/) — Order execution on exchanges
- **Schemas** (schemas/) — Pydantic message models
- **Topics** (topics/) — ZMQ topic definitions

Message types (in messaging.schemas.data):

- TickData — Price tick
- CandleData — OHLCV candle
- SignalData — Trading signal
- TradeData — Trade execution
- OrderData — Order status
- ExecutionData — Order execution
- OrderRequestData — Order request
- HeartbeatData — Component heartbeat

Every Data class carries `public_id: str (UUID7)`, `type: Literal[...]`, and `timestamp: datetime`.

Application (src/snapper/application/)

Business logic:

- **Engine** (engine/) — TraderCoordinator is the multi-symbol trade runtime coordinator; TradingEngineService is the per-instrument engine that turns signals into TradeCommand rows and order requests
- **Trade** (trade/) — Trade-domain services: trade_service.py (in-memory command and position read model), balance_service.py (cash/equity/exposure projection), outbox.py (outbox-driven publishing), reconciler.py (stale-command scan and reconciliation failure feedback)
- **Process Manager** (process_manager/) — Process management
- **Services** (services/) — Application services
- **Updaters** (updaters/) — Data updates (symbols, historical)
- **Risk** (risk/) — Risk management
- **Portfolio** (portfolio/) — Portfolio management

Infrastructure (src/snapper/infrastructure/)

External integrations:

- **Exchanges** (exchanges/) — Exchange clients (Kraken, Zonda, Walutomat)
- **Market Data** (market_data/) — WebSocket feeds
- **Symbols** (symbols/) — Symbol mapping
- **Security** (security/) — Settings encryption
- **Logging** (logging/) — Logging configuration

Server (src/snapper/server/)

FastAPI application:

- **App Factory** (app.py) — create_app() with lifespan management
- **REST API** — HTTP endpoints
- **WebSocket** — Real-time streaming via ZMQ bridge
- **Static Files** — Frontend dashboard
- **ClientProvenanceMiddleware** (provenance_middleware.py) — Extracts client provenance from mutation requests, runs per-session gap detection, records mutations to control table and GET reads to telemetry table

API (src/snapper/api/)

Shared API schemas and WebSocket auth helpers (not route definitions):

- **Schemas** (schemas/) — Pydantic request/response models (health, process, settings).

Schema class hierarchy:

```
text
BaseModel
├── PartialBody          - extra="ignore", strict=True (GapEnvelope,
WsTokenPayload)
├── StrictBody           - extra="forbid", strict=True (request bodies,
nested DTOs)
│   ├── StrictDataSchema - + provenance (all event payloads, REST/WS/ZMQ)
│   │   ├── PayloadRequest[T]
│   │   ├── PayloadResponse[T]
│   │   └── PayloadListResponse[T]
├── ExchangeResponse     - extra="allow", strict=True (exchange API
parsing)
├── ExchangeRequest      - extra="forbid", strict=True (outgoing exchange
requests)
└── BaseSettings         - BootstrapSettingsLoader (env var config)
```

- **Auth** (auth/) — WebSocket token service and schemas

Route modules live closer to their domains: `server/app.py` (assembly and data endpoints), `server/process_routes.py`, `server/strategy_routes.py`, `config/settings_routes.py`, and `auth/routes.py`.

REST endpoints return the same Data schemas used by WebSocket messaging (`messaging.schemas.data`): `OrderData`, `SignalData`, `ExecutionData`, `PositionData`, `CandleData`. Old REST-specific schema modules were removed.

Auth (`src/snapper/auth/`)

Authentication system:

- JWT tokens with access/refresh via HTTP-only cookies
- CSRF protection
- Role-based access (viewer, operator, admin)
- WebSocket authentication

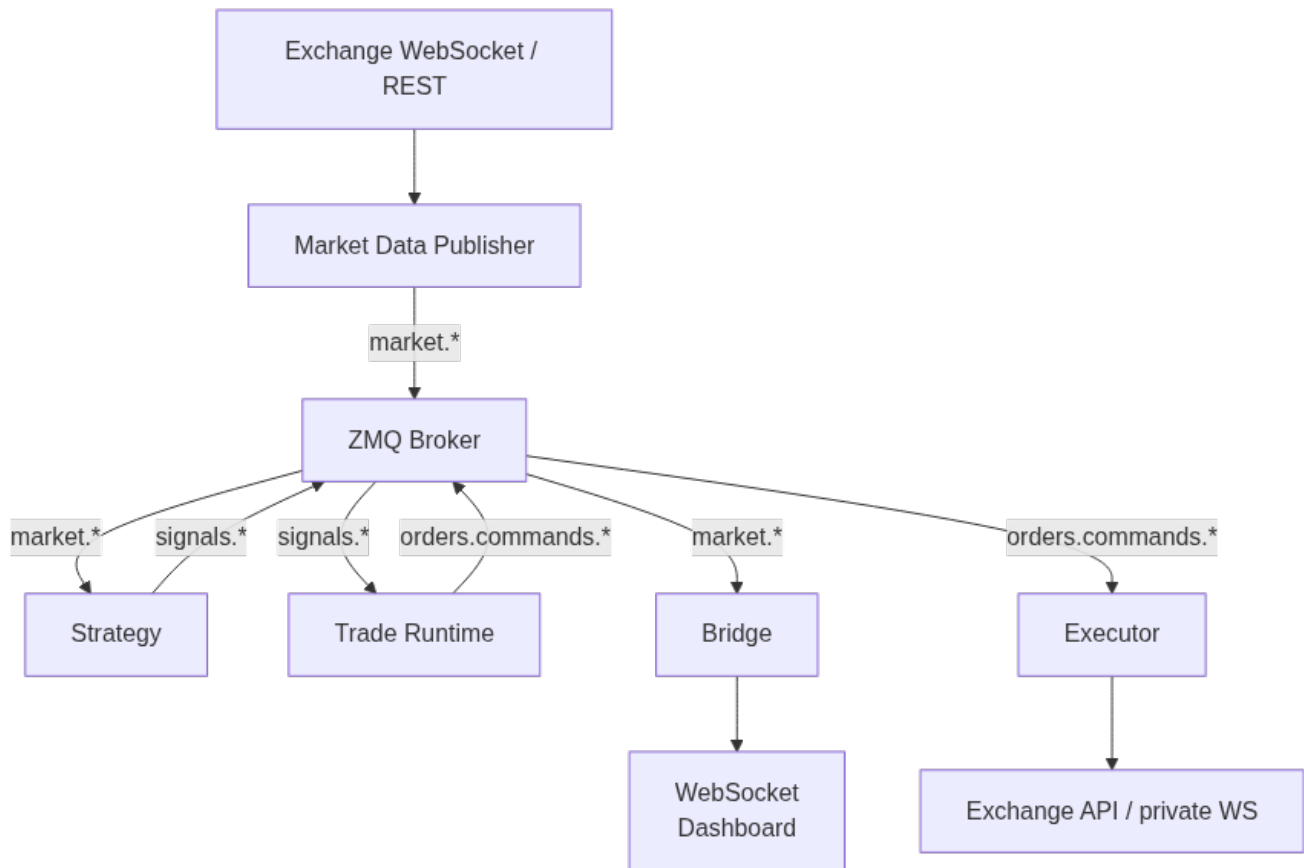
CLI (`src/snapper/cli/`)

Command-line interface (Typer):

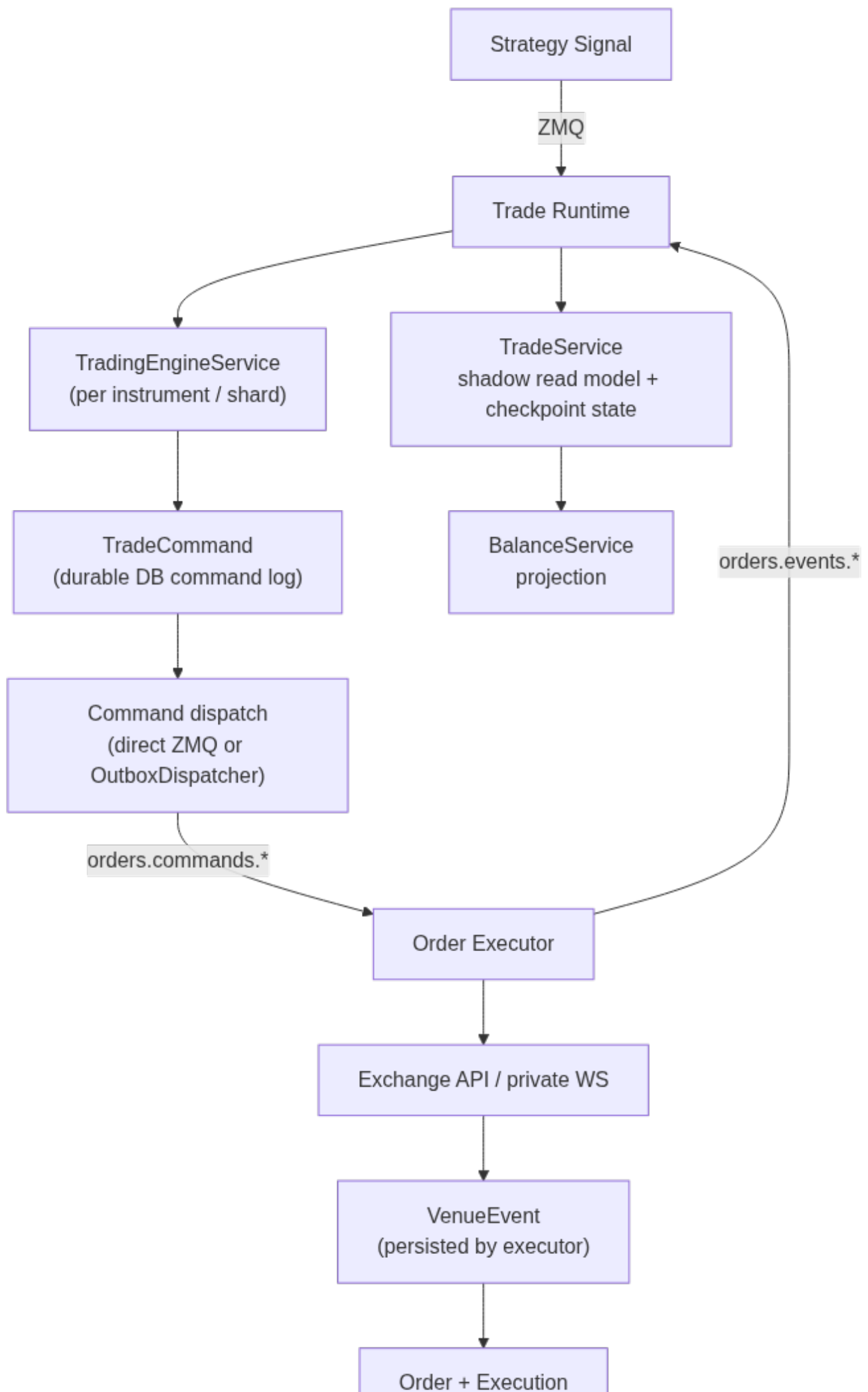
- Server and infrastructure management
- Database operations
- User management
- Market data updates

Data Flow

Market Data and Signal Flow



Order Flow



Database

SQLite for development, PostgreSQL for production.

Schema

```
-- Market data (joined to instruments via instrument_public_id)
instruments      -- Financial instruments (logical key: symbol_public_id +
exchange)
candles          -- OHLCV data
ticks            -- Real-time price snapshots
trades           -- Transaction history
market_snapshots -- Real-time market data (SCD2 per instrument, one active
row each)

-- Trading (joined to instruments via instrument_public_id)
orders           -- Order history (mode: live/paper)
executions       -- Order executions (joined to orders via order_public_id)
positions        -- Portfolio positions (mode: live/paper)
signals          -- Strategy signals

-- Trade runtime (durable command path)
trade_commands   -- Durable trade intent (engine writes before
execution)
venue_events     -- Durable venue observations and acknowledgements
from executors
trade_projection_checkpoints -- Materialized position/balance snapshots

-- Symbol management
symbols          -- Symbol identity + versioned attributes (SCD2)
symbol_aliases   -- Exchange-specific symbol mappings
symbol_exchange_capabilities -- Exchange-specific capabilities

-- Instrument specifications
instrument_specs  -- Trading specifications (tick_size, lot_size, limits)

-- System
users            -- User accounts
settings         -- Settings (encrypted)
user_login_events -- Authentication event log
process_runs     -- Background process execution records
control          -- Audit log (commands, auth events, mutations)
telemetry        -- High-volume operational events (toggleable)
```

Migrations

Alembic manages migrations:

```
snapper db-upgrade  # Apply migrations
snapper db-downgrade # Rollback
```

Trade Runtime

The trade runtime supports two deployment modes controlled by the `use_durable_commands` database setting (default: false):

- **Dual-write** (default) — `TradingEngineService` writes `TradeCommand` rows and also publishes directly to ZMQ. Durable tables are populated in the background, and `direct_dispatched` prevents the outbox from replaying already sent commands.
- **Durable** (outbox-driven) — `TradingEngineService` writes `TradeCommand` rows only. `OutboxDispatcher` publishes from the database, and executor `VenueEvent` writes become fail-closed on the accepted/fill paths.

The `TraderCoordinator` class acts as the trade runtime coordinator and integrates `TradeService` (command lifecycle) and `BalanceService` (balance tracking) in both modes. When `use_durable_commands=true`, it also starts the outbox dispatcher and the reconciliation/circuit-breaker task. In dual-write mode, the runtime still writes `TradeCommand` rows, consumes `orders.events.*` to shadow-update trade projections, and persists checkpoints for recovery. Canonical Order and Execution rows are persisted on the exchange/executor path. Restart the trade runtime and executors after changing `use_durable_commands`.

Bitemporal Model

All entity tables inherit `TemporalMixin` which provides `id`, `public_id`, `timestamp` (bus-time / `known_from`), and `known_to` columns.

Key concepts:

- **SCD Type 2** — No in-place UPDATE or DELETE. Mutations use a close+insert pattern: the current row is closed (`known_to` set to now) and a new row is inserted with the updated values and `known_to = KNOWN_TO_MAX`.
- **KNOWN_TO_MAX** — Sentinel value (9999-12-31T23:59:59 UTC) marking the active version of a row.
- **Point-in-time queries** — REST endpoints accept an `as_of` query parameter to retrieve the state of data at a specific moment.
- **Helpers:**
 - `where_active(model, at)` — SQLAlchemy filter for temporal queries. `at`: datetime is required — no silent default-to-now
 - `where_active_now(model)` — convenience for operations that genuinely mean "current state" (auth login, heartbeat). Not a shortcut for skipping `as_of` threading
 - `close_and_insert()` / `close_and_insert_sync()` — Repository helpers that atomically close the current row and insert a new version

Processes

The system manages processes through Process Manager:

- **Core** — Broker, Feed, Bridge (required — startup is aborted if any enabled long-running CORE process fails to start)
- **Strategy** — Trading strategies
- **Task** — One-time tasks
- **Backtest** — Backtesting processes

Processes can be:

- `long_running` — Run continuously (services)
- `one_shot` — Execute once (tasks)

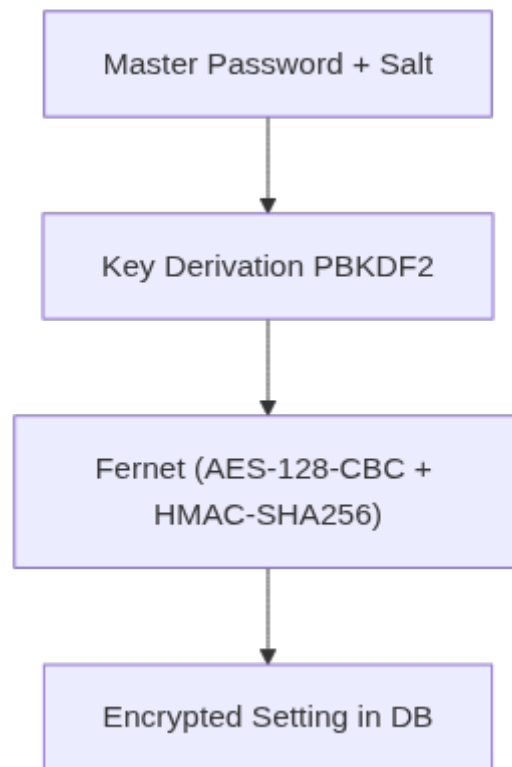
Deployment Modes

By default, the FastAPI lifespan starts all enabled processes (all-in-one dev mode). Set `SERVER_API_ONLY=true` to skip process autostart -- the ZMQ-WebSocket bridge still starts, so the frontend receives live data from a separately-running engine. Useful for multi-worker uvicorn or when broker/strategies/executors run on different hosts.

Security

Settings Encryption

Sensitive data (API keys) are encrypted in the database:



Authentication

- JWT tokens via HTTP-only cookies (access: 15min, refresh: 7 days)
- CSRF tokens for mutating requests
- WebSocket tokens (one-time)
- Bcrypt password hashing

CLI

Snapper provides a command-line interface built on the Typer framework. All commands are available through the snapper command.

Usage

```
snapper [COMMAND] [OPTIONS]
```

Help for a specific command:

```
snapper [COMMAND] --help
```

Server and Infrastructure

server

Starts the FastAPI server with the web dashboard.

```
snapper server [OPTIONS]
```

Options:

Option	Type	Default	Description
--host	string	from env	Listen address
--port	int	from env	Server port
--reload	bool	from env	Auto-reload for development

Environment:

Set `SERVER_API_ONLY=true` to skip process autostart while keeping the ZMQ-WebSocket bridge alive (useful for multi-worker deployments or when the engine runs as a separate process).

Examples:

```
# Start with default settings (all-in-one dev mode)
snapper server

# Start on all interfaces
```

```
snapper server --host 0.0.0.0 --port 8000

# Development mode with auto-reload
snapper server --reload

# API-only mode (engine runs separately)
SERVER_API_ONLY=true snapper server
```

broker

Starts the ZeroMQ XPUB/XSUB broker for message routing.

```
snapper broker [OPTIONS]
```

Options:

Option	Type	Default	Description
--xsub	string	from env	XSUB endpoint (publishers)
--xpub	string	from env	XPUB endpoint (subscribers)

Example:

```
snapper broker --xsub tcp://127.0.0.1:7500 --xpub tcp://127.0.0.1:7501
```

trade-zmq

Starts the central trade runtime coordinator.

The process still runs as the trade-zmq command, but after the trade runtime redesign it also hosts TradeService, BalanceService, and, when durable mode is enabled, the outbox dispatcher and reconciliation loop. In dual-write mode the same process still writes TradeCommand rows, consumes orders.events.* to keep projections in sync, and persists checkpoints for recovery.

```
snapper trade-zmq [OPTIONS]
```

Options:

Option	Type	Default	Description
--signal-topics	string	signals.	Signal topics to subscribe

Example:

```
snapper trade-zmq --signal-topics "signals.paper.BTC-USD.rsi_btc_1h,signals.kraken.BTC-USD.live"
```

executor

Starts the order execution service for a selected exchange.

```
snapper executor [OPTIONS]
```

Options:

Option	Type	Default	Description
-e, --exchange	string	kraken	Exchange (kraken, zonda, walutomat)

Examples:

```
# Kraken executor
snapper executor -e kraken

# Zonda executor
snapper executor --exchange zonda
```

feed

Starts the market data publisher via WebSocket.

```
snapper feed [OPTIONS]
```

Options:

Option	Type	Default	Description
--symbols	string	BTC-USD	Comma-separated symbols

Example:

```
snapper feed --symbols "BTC-USD,ETH-USD,SOL-USD"
```

zmq-logger

Monitors and logs ZMQ message traffic.

```
snapper zmq-logger [OPTIONS]
```

Options:

Option	Type	Default	Description
--payload/--no-payload	bool	false	Log full payload
--file/--no-file	bool	true	Write to audit file
--audit-file	string	data/zmq_audit.jsonl	Audit file path
--max-length	int	200	Max payload preview length

Example:

```
snapper zmq-logger --payload --audit-file logs/zmq.jsonl
```

Database

db-init

Initializes the database schema.

```
snapper db-init
```

db-upgrade

Runs Alembic migrations to a specified revision.

```
snapper db-upgrade [OPTIONS]
```

Options:

Option	Type	Default	Description
--revision	string	head	Target revision

Examples:

```
# Migrate to latest version
snapper db-upgrade

# Migrate to specific revision
snapper db-upgrade --revision abc123
```

db-downgrade

Rolls back Alembic migrations.

```
snapper db-downgrade [OPTIONS]
```

Options:

Option	Type	Default	Description
--revision	string	-1	Steps to roll back

Examples:

```
# Roll back one migration
snapper db-downgrade

# Roll back to specific revision
snapper db-downgrade --revision base
```

db-seed

Seeds the database with environment-specific data from TOML profiles. Automatically run by make migrate-dev and make migrate-prod.

```
snapper db-seed [OPTIONS]
```

Options:

Option	Type	Default	Description
--profile	string	dev	Seed profile name (e.g. dev, prod)

Users

init-admin

Creates the initial administrator user.

```
snapper init-admin [OPTIONS]
```

Options:

Option	Type	Default	Description
--username	string	admin	Username
--password	string	AdminSnapper2026!	Password

Example:

```
snapper init-admin --username admin --password SuperSecurePass123!
```

list-users

Displays a list of all system users.

```
snapper list-users
```

reset-password

Resets a user's password.

```
snapper reset-password USERNAME [OPTIONS]
```

Arguments:

- USERNAME — Username (required)

Options:

Option	Type	Default	Description
--new-password	string	(prompt)	New password

Example:

```
snapper reset-password john --new-password NewSecurePass123!
```

Symbol Synchronization

update-kraken-symbols

Synchronizes symbol mappings from the Kraken API.

```
snapper update-kraken-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-zonda-symbols

Synchronizes symbol mappings from the Zonda API.

```
snapper update-zonda-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-walutomat-symbols

Synchronizes symbol mappings from the Walutomat API.

```
snapper update-walutomat-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-polygon-symbols

Synchronizes symbol mappings from the Polygon.io API.

```
snapper update-polygon-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update
--insert-new	bool	false	Insert new symbols

Note: This operation may take 10-15 minutes (44k+ symbols).

Market Snapshots

update-kraken-market-snapshot

Updates Kraken market snapshots with current prices.

```
snapper update-kraken-market-snapshot
```

update-zonda-market-snapshot

Updates Zonda market snapshots with current prices.

```
snapper update-zonda-market-snapshot
```

update-walutomat-market-snapshot

Updates Walutomat market snapshots with current prices.

```
snapper update-walutomat-market-snapshot
```

Data Backfill

polygon-backfill-aggregates

Fetches historical aggregated data from the Polygon.io API.

```
snapper polygon-backfill-aggregates [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	string[]	from settings	Symbols to backfill
--all	bool	false	All mapped symbols
-m, --multiplier	int	1	Timeframe multiplier
-t, --timespan	string	minute	Timespan (minute, hour, day)
-d, --days	int	from settings	Days back

Option	Type	Default	Description
--resume/--no-resume	bool	true	Resume from last timestamp
--csv/--no-csv	bool	true	Save to CSV.gz files

Examples:

```
# Backfill for specific symbols
snapper polygon-backfill-aggregates -s AAPL -s MSFT --days 365

# Backfill all mapped symbols
snapper polygon-backfill-aggregates --all --timespan day

# Hourly backfill with resume
snapper polygon-backfill-aggregates -s SPY -t hour -m 1 --resume
```

polygon-backfill-grouped

Fetches grouped daily data from the Polygon.io API.

```
snapper polygon-backfill-grouped [OPTIONS]
```

Data Archive

archive

Export data to CSV archive files. Supports candle cache projection and append-only event tables (ticks, trades, signals, executions, telemetry, control). Merges with existing files and deduplicates rows.

```
snapper archive [OPTIONS]
```

Options:

Option	Type	Default	Description
--table	TEXT	candles	Table to archive (candles, candles-audit, ticks, trades, signals, executions, telemetry, control)
--exchange	TEXT	None	Exchange filter (e.g. polygon, kraken)
--symbol	TEXT	None	Native symbol filter (e.g. BTC-USD), resolved to stable archive_symbol
--timeframe	TEXT	1m	Candle timeframe (1m, 5m, 1h, 1d) — candles only

Option	Type	Default	Description
--day	DATE	None	Single day to archive (YYYY-MM-DD)
--from	DATE	None	Start of date range (YYYY-MM-DD)
--to	DATE	None	End of date range (YYYY-MM-DD)
--dry-run	FLAG	False	Report counts without writing files
--purge	FLAG	False	Delete exported rows from DB after writing (event tables and candles-audit with --closed-only)
--closed-only	FLAG	False	Only closed SCD2 versions (candles-audit only)
--output-dir	PATH	data	Base output directory

Output structure:

```
Candles: {output_dir}/{exchange}/cache/{timespan}/{archive_symbol}/{year}/{date}.csv
Events:  {output_dir}/archive/{table}/{exchange}/{archive_symbol}/{year}/{date}.csv
Flat:    {output_dir}/archive/{table}/{year}/{date}.csv  (telemetry, control)
```

Examples:

```
snapper archive --day 2024-03-15 --exchange polygon --symbol BTC-USD
snapper archive --from 2024-01-01 --to 2024-03-31 --dry-run
snapper archive --day 2024-03-15 --timeframe 1d
snapper archive --table ticks --day 2024-03-15 --exchange polygon
snapper archive --table trades --from 2024-01-01 --to 2024-03-31 --purge
snapper archive --table control --day 2024-03-15
snapper archive --table candles-audit --day 2024-03-15 --timeframe 1m
snapper archive --table candles-audit --day 2024-03-15 --closed-only --purge
snapper archive --table orders --day 2024-03-15
snapper archive --table symbols --day 2024-03-15 --closed-only --purge
snapper archive --table instruments --day 2024-03-15
```

Notes:

- --symbol resolves the current native_symbol to the stable archive_symbol via the Symbol anchor row. After a symbol rename, the CLI argument and the output directory may differ.
- --day or --from/--to is required.
- Daily timespan (1d) produces monthly CSV files to match Polygon layout.
- --purge is only supported for event tables, not candle cache export.
- Event archive CSV files include full temporal metadata (public_id, timestamp, known_to, session_id, sequence_id).
- Merge/dedup for events uses (public_id, timestamp, known_to) as key.

- Executions are partitioned by exchange/archive_symbol via order -> instrument.
- candles-audit exports all SCD2 versions (closed + active) grouped by open_at date. --purge requires --closed-only to protect active rows.

restore

Restore archived CSV data back into the database. Reads CSV files exported by snapper archive and inserts rows, deduplicating against existing data by (public_id, timestamp, known_to).

```
snapper restore [OPTIONS]
```

Options:

Option	Type	Default	Description
--table	TEXT	(required)	Table name to restore into
--file	PATH	None	Single CSV file to restore
--dir	PATH	None	Directory to scan recursively for CSV files
--source	TEXT	audit	Restore mode (audit for full history)

Examples:

```
snapper restore --table ticks --file data/archive/ticks/polygon/BTC-USD/2024/2024-01-01.csv
snapper restore --table settings --dir data/archive/settings/
snapper restore --table candles --dir data/archive/candles/polygon/BTC-USD/
```

Notes:

- At least one of --file or --dir is required.
- Rows already present in DB (matching public_id + timestamp + known_to) are skipped.
- Audit restore inserts all temporal columns exactly as exported.

One-time cache directory migration

Renames legacy Polygon cache directories from API ticker format (X_BTCUSD) to archive_symbol format (BTC-USD).

```
python -m scripts.migrate_polygon_cache_dirs [--dry-run] [--cache-root PATH]
```

Encryption Management

settings-rotate-encryption

Rotates encryption keys for settings in the database.

```
snapper settings-rotate-encryption [OPTIONS]
```

Options:

Option	Type	Default	Description
--new-password	string	(required)	New master password
--old-password	string	from env	Current master password (optional, reads from env/bootstrap)
--dry-run	bool	false	Show changes without applying

Example:

```
# Simulate rotation
snapper settings-rotate-encryption --new-password NewSecurePass --dry-run

# Actual rotation
snapper settings-rotate-encryption --new-password NewSecurePass
```

Example Workflows

New Installation Initialization

```
# 1. Initialize database
snapper db-init

# 2. Create administrator
snapper init-admin --username admin --password SecretPassword123!

# 3. Synchronize symbols
snapper update-kraken-symbols
snapper update-polygon-symbols

# 4. Start server
snapper server
```

Full Trading System Startup

```
# Terminal 1: ZMQ Broker
snapper broker

# Terminal 2: Data feed
snapper feed --symbols "BTC-USD,ETH-USD"

# Terminal 3: Executor
snapper executor -e kraken

# Terminal 4: Trade runtime
snapper trade-zmq

# Terminal 5: Server with dashboard
SERVER_API_ONLY=true snapper server
```

Historical Data Backfill

```
# Backfill last 30 days for SPY
snapper polygon-backfill-aggregates -s SPY --days 30 --timespan minute

# Backfill daily data for all symbols
snapper polygon-backfill-aggregates --all --timespan day --days 365
```

Configuration

Snapper's configuration system is based on two layers: environment variables (bootstrap) and database settings.

Environment Variables

Settings loaded from .env file or environment variables.

Database

Variable	Default	Description
DB_URL	sqlite+aiosqlite:///./data/snapper.db	SQLAlchemy connection URL

Database URL examples:

```
# SQLite (development default)
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# PostgreSQL (production)
DB_URL=postgresql+asyncpg://user:password@localhost:5432/snapper
```

Encryption

Variable	Default	Description
MASTER_PASSWORD	snapper_default_master_password_v1	Password for encrypting settings in database (salt derived automatically)

Important: Change default values in production environment.

HTTP Server

Variable	Default	Description
SERVER_HOST	127.0.0.1	Listen address
SERVER_PORT	8000	Server port
SERVER_RELOAD	false	Auto-reload for development

Variable	Default	Description
SERVER_PROXY_HEADERS	true	Enable proxy header parsing in uvicorn
SERVER_FORWARDED_ALLOW_IPS	127.0.0.1	Trusted proxy IPs/CIDRs for forwarded headers
SERVER_API_ONLY	false	Skip process autostart; serve API + WS bridge only (for multi-worker or separate engine)
TELEMETRY_RECORDING_ENABLED	false	Record pings, heartbeats, and GET reads to telemetry table. High volume — enable for debugging only

ZeroMQ

Variable	Default	Description
ZMQ_BROKER_XSUB	tcp://127.0.0.1:7500	XSUB endpoint (publishers connect here)
ZMQ_BROKER_XPUB	tcp://127.0.0.1:7501	XPUB endpoint (subscribers connect here)

Database Settings

Sensitive data is stored encrypted in the settings table.

Exchange API Keys

Key	Description
kraken_api_key	Kraken API key
kraken_api_secret	Kraken API secret
polygon_api_key	Polygon.io API key
walumat_api_key	Walumat API key
walumat_private_key	Walumat private key
zonda_api_key	Zonda API key
zonda_api_secret	Zonda API secret

Trading Parameters

Key	Default	Description
instruments	{"kraken": ["BTC-USD", ...], ...}	Instruments per exchange (dict)
timeframes	["1m"]	Candle timeframes (list)
risk_max_leverage	1.0	Maximum leverage
risk_r_per_trade	0.005	Risk per trade (0.5%)
use_durable_commands	false	Enable durable (outbox-driven) trade runtime instead of dual-write

use_durable_commands is read by the trade runtime and executors during startup. After changing it, restart snapper trade-zmq and the relevant snapper executor processes.

Mode summary:

- false — dual-write mode. The engine writes durable command rows and still publishes directly to ZMQ.
- true — outbox-driven durable mode. Commands are published from the database outbox and executor VenueEvent writes become fail-closed.

Publisher Micro-Batch

Key	Default	Description
write_buffer_flush_ms	50	Flush age threshold in ms (cached at start)
write_buffer_candle_max_rows	100	Candle batch size trigger
write_buffer_tick_max_rows	500	Tick batch size trigger
write_buffer_trade_max_rows	500	Trade batch size trigger

These settings are cached when the publisher starts and require a process restart to take effect.

Authentication

Key	Default	Description
auth_secret_key	change-me-in-production-use-openssl-rand-hex-32	JWT signing secret

Key	Default	Description
csrf_secret_key	change-me-in-production-csrf-key	CSRF token signing secret
auth_algorithm	HS256	JWT signing algorithm
auth_access_token_expire_minutes	15	Access token lifetime
auth_refresh_token_expire_days	7	Refresh token lifetime
auth_refresh_token_expire_days_extended	30	Extended refresh token lifetime
ws_token_ttl_seconds	900	WebSocket token lifetime
csrf_token_expire_minutes	60	CSRF token lifetime
session_secure	false	Require HTTPS for session cookies
session_same_site	lax	Session cookie SameSite mode
session_domain	""	Session cookie domain override
ui_origin	""	Additional allowed UI origins for CORS and WS origin validation

.env.example File

```

# Database connection
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# Encryption settings for sensitive data (API keys, secrets)
# IMPORTANT: Change this default in production!
MASTER_PASSWORD=snapper_default_master_password_v1

# Server settings
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=false
SERVER_API_ONLY=false
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1

# ZMQ broker settings
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501

```

Accessing Configuration in Code

Bootstrap settings (without database)

```
from snapper.config.settings import get_settings

settings = get_settings()
db_url = settings.db_url
host = settings.server_host
```

With database access

```
from snapper.config.settings import get_settings_with_service

settings = get_settings_with_service(settings_service)
api_key = settings.kraken_api_key # Decrypted from database
```

Managing Settings via API

List Settings

```
GET /api/settings
GET /api/settings?category=exchange
```

List Categories

```
GET /api/settings/categories
```

Set Setting

```
POST /api/settings/{key}/set
X-CSRF-Token: <csrf_token>
Content-Type: application/json

{
  "public_id": "<uuid7>",
  "session_id": "<client-session>",
  "sequence_id": 1,
  "timestamp": "2026-01-18T12:00:00Z",
  "payload": {
    "value": "new_value",
    "category": "system",
```

```
    "description": "optional description"
  }
}
```

Remove Setting

```
POST /api/settings/{key}/remove
X-CSRF-Token: <csrf_token>
Content-Type: application/json

{
  "public_id": "<uuid7>",
  "session_id": "<client-session>",
  "sequence_id": 2,
  "timestamp": "2026-01-18T12:00:01Z",
  "payload": {}
}
```

Environment Configuration

Development

```
# .env.development
DB_URL=sqlite+aiosqlite:///./data/snapper.db
SERVER_HOST=127.0.0.1
SERVER_PORT=8000
SERVER_RELOAD=true
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1
```

Production

```
# .env.production
DB_URL=postgresql+asyncpg://user:pass@db.example.com/snapper
MASTER_PASSWORD=<strong_random_password>
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=false
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1
```

Docker

Variables can be passed via docker-compose.yml:

```
services:
  snapper:
    env_file: .env
    environment:
      - DB_URL=postgresql+asyncpg://snapper:password@postgres/snapper
```

Settings Encryption

Sensitive settings are encrypted with Fernet (AES-128-CBC + HMAC-SHA256):

1. Salt is derived from MASTER_PASSWORD via SHA-256
2. Encryption key is derived via PBKDF2-HMAC-SHA256 (100,000 iterations)
3. Values are encrypted with Fernet and stored in database

If MASTER_PASSWORD changes, encrypted settings become inaccessible. Use snapper settings-rotate-encryption to rotate passwords safely.

Configuration Validation

The system validates configuration at startup:

- Database connection check
- ZMQ endpoint verification
- Settings decryption test

Configuration errors are reported through exceptions with clear messages.

Development

Guidelines for developers working on the Snapper project.

Requirements

- Python 3.14+
- Poetry
- Node.js 25+ and pnpm
- TA-Lib (C library)
- Pre-commit hooks

Environment Setup

```
# Install system dependencies (macOS)
make system-deps

# Install Python dependencies
make setup

# Install frontend dependencies
make ui-setup

# Synchronize pre-commit hooks
make pre-refresh
```

Quality Gates

Main Command

```
make check-all
```

Executes the complete quality gate:

1. Formatting (ruff, black, isort)
2. Linting (ruff)
3. Type checking (mypy)
4. Docstring compliance
5. No # comments in Python
6. Canonical `__main__` guards

7. Empty `__init__.py` files
8. No forbidden temporal mutations
9. No pragma/noqa/ignore exclusions
10. Tests with 100% coverage
11. Frontend (ESLint, Prettier, dead code, tests with coverage)

Individual Steps

Command	Description
make fmt	Check formatting
make fmt-fix	Fix formatting
make lint	Ruff linting
make lint-fix	Fix linting issues
make typecheck	Mypy type checking
make test	Tests (parallel)
make test-serial	Tests (sequential, debug)
make cov	Tests with coverage

Automatic Fixes

```
make fix          # Backend: fmt-fix + lint-fix + move-imports
make fix-all      # Backend + frontend
```

Code Standards

Types (MANDATORY)

All functions must have type annotations:

```
def calculate_rsi(prices: pd.Series, period: int = 14) -> pd.Series:
    """Calculate RSI indicator."""
    ...

async def fetch_data(symbol: str) -> list[Candle]:
    """Fetch candle data."""
    ...
```

Test functions must have `-> None`:

```
async def test_rsi_calculation() -> None:
    """Test RSI calculation returns expected values."""
    ...
```

Docstrings (Google style)

```
def process_signal(signal: Signal, config: dict[str, Any]) -> Order | None:
    """Process trading signal and generate order.

    Takes a trading signal and configuration, validates the signal,
    and generates an appropriate order if conditions are met.

    Args:
        signal: Trading signal to process.
        config: Strategy configuration dictionary.

    Returns:
        Generated order if valid, None otherwise.

    Raises:
        ValueError: If signal data is invalid.
    """
    ...
```

No # Comments

comments are forbidden. Use docstrings:

```
# WRONG
def foo():
    # This calculates something
    x = 1 + 1

# CORRECT
def foo():
    """Calculate the sum."""
    x = 1 + 1
```

Imports

Imports at the top of file, single lines:

```
from datetime import UTC
from datetime import datetime
from typing import Any

import pandas as pd
from sqlalchemy import select
```

```
from snapper.config.settings import get_settings
from snapper.data.models import Candle
```

Forbidden:

- from __future__ import annotations
- if TYPE_CHECKING: blocks
- Re-exports in __init__.py

__init__.py

__init__.py files must be empty (or docstring only):

```
"""Snapper strategies module."""
```

Tests

Structure

```
tests/
├── conftest.py           # Shared fixtures
├── api/                  # API tests
├── application/          # Business logic tests
├── cli/                  # CLI tests
├── data/                 # Data layer tests
├── indicators/           # Indicator tests
├── messaging/            # ZMQ tests
├── strategies/           # Strategy tests
└── ...
```

Writing Tests

```
import pytest
from snapper.indicators.rsi import rsi

class TestRSI:
    """Test suite for RSI indicator."""

    def test_rsi_returns_series_with_correct_length(self) -> None:
        """
        Given a price series of length N
        When RSI is calculated with period P
        Then the result has length N.
        """
        prices = pd.Series([100, 102, 101, 103, 105])
        result = rsi(prices, period=14)
```

```

        assert len(result) == len(prices)

def test_rsi_values_in_valid_range(self) -> None:
    """
    Given any price series
    When RSI is calculated
    Then all values are between 0 and 100.
    """
    prices = pd.Series([100, 102, 101, 103, 105, 104, 106])
    result = rsi(prices, period=3)
    assert (result >= 0).all()
    assert (result <= 100).all()

```

Async Tests

```

import pytest

@pytest.mark.asyncio
async def test_fetch_candles() -> None:
    """Test fetching candles from repository."""
    repo = get_repository(db_url)
    candles = await repo.get_candles("BTC-USD", limit=10)
    assert len(candles) <= 10

```

Fixtures

```

@pytest.fixture
def sample_candles() -> list[Candle]:
    """Provide sample candle data for tests."""
    return [
        Candle(open=100, high=105, low=95, close=102, volume=1000),
        Candle(open=102, high=110, low=100, close=108, volume=1200),
    ]

```

100% Coverage

The project requires 100% code coverage (TDD). Check:

```
make cov
```

Configuration in .coveragerc.

Frontend

Setup

```
make ui-setup
```

Development

```
make ui-dev
```

Frontend at <http://localhost:3000/> with backend proxy.

Quality Checks

Command	Description
make ui-lint	ESLint
make ui-format	Prettier check
make ui-dead-code	Knip dead code
make ui-test	Vitest tests
make ui-cov	Tests with coverage

Fixes

```
make ui-fix    # lint-fix + format-fix + dead-code-fix
```

Type Generation

```
make ui-gen-types  # OpenAPI types, WebSocket types, Zod schemas, entity  
aliases, permissions
```

Database Migrations

Dev DB Location

The development SQLite database lives at `./data/snapper.db`.

Destructive Migration Strategy (Development)

The project uses a single-file migration (0001_init.py) that is rewritten in place when the schema changes. After editing the migration, reset and reseed:

```
rm -f data/snapper.db && make migrate-dev
```

make migrate-dev applies Alembic migrations (via snapper db-init) then seeds development data (via snapper db-seed --profile dev). There is no standalone make migrate target -- always use make migrate-dev or make migrate-prod.

Applying (Production)

```
make migrate-prod
```

Rollback

```
snapper db-downgrade
```

Docker

Build

```
make docker-build-dev    # Development image  
make docker-build-prod   # Production image
```

Run

```
make docker-run
```

Docker Compose

```
docker compose up -d
```

CI/CD

Pipeline executes the same consolidated gate used locally:

1. make check-all — Backend checks, frontend checks, exclusion scan, and tests with coverage

For debugging a failing pipeline locally, the equivalent steps are:

1. make check — Backend quality checks
2. make ui-check — Frontend lint, format, and dead-code checks
3. make check-exclusions — No pragma/noqa/ignore bypasses
4. make cov — Backend tests with coverage
5. make ui-cov — Frontend tests with coverage

Workflow

Before Commit

```
make check-all
```

Before PR

1. Ensure make check-all passes
2. Add/update tests
3. Update documentation if needed
4. Verify 100% coverage

Code Review Checklist

- [] Types for all functions
- [] Docstrings (Google style)
- [] No # comments
- [] Tests for new functionality
- [] 100% coverage
- [] make check-all passes

Project Structure

```
snapper/
├── src/snapper/           # Source code
│   ├── api/              # API schemas
│   ├── application/      # Business logic
│   ├── auth/             # Authentication
│   ├── cli/              # CLI interface
│   ├── config/           # Configuration
│   ├── core/             # Core types
│   ├── data/             # Data layer
│   ├── indicators/       # Technical indicators
│   ├── infrastructure/   # External integrations
│   ├── interface/       # WebSocket
│   ├── messaging/        # ZeroMQ
│   ├── server/           # FastAPI
│   ├── strategies/       # Trading strategies
│   └── utils/            # Utilities
├── tests/                # Tests
├── frontend/             # React dashboard
├── docs/                 # Documentation
├── scripts/              # Helper scripts
└── data/                 # Local data
```

Tools

Ruff

Linters and formatters. Configuration in pyproject.toml:

```
[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP", "C4", "SIM", "N", "PERF", "ISC",
"PLW", "PLC", "A", "TID", "D"]
```

Mypy

Type checker. Strict mode enabled:

```
[tool.mypy]
python_version = "3.14"
strict = true
```

Black

Formatter. Line length 100:

```
[tool.black]
line-length = 100
```

Pre-commit

Hooks run before commit:

```
pre-commit install
pre-commit run --all-files
```

Debugging

Logging

```
from loguru import logger

logger.info(f"Processing signal: {signal}")
logger.debug(f"Calculated RSI: {rsi_value}")
logger.error(f"Failed to execute order: {e}")
```

ZMQ Logger

```
snapper zmq-logger --payload
```

Serial Tests

```
make test-serial
```

pdb/ipdb

```
import ipdb; ipdb.set_trace()
```

Documentation

Docstrings

Every module, class, and public function must have a docstring.

docs/

Markdown documentation in docs/ directory.

PDF

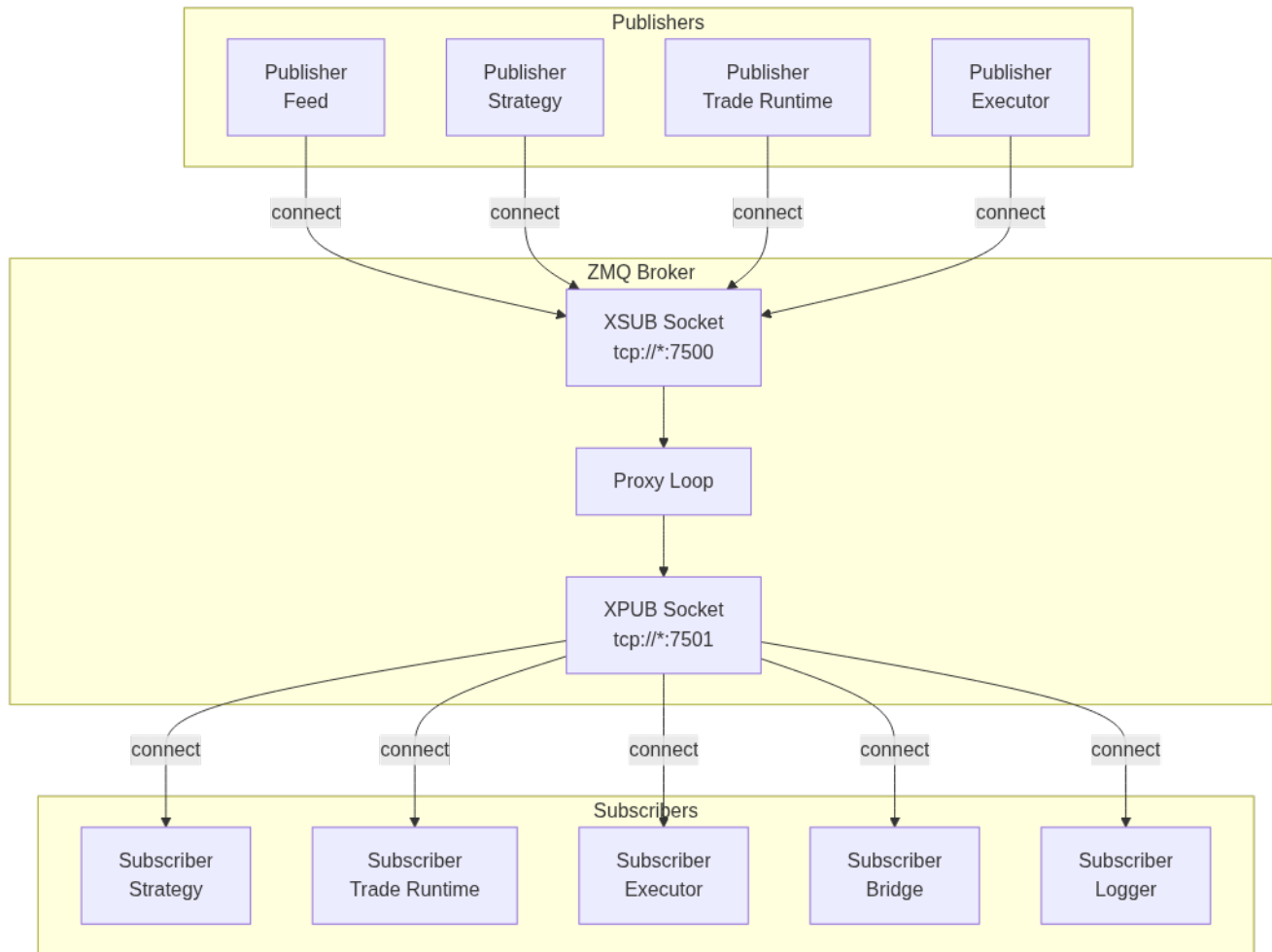
```
make docs-pdf
```

Generates snapper.pdf from README + docs/*.md.

Messaging (ZeroMQ)

Snapper uses ZeroMQ for inter-process communication in a pub/sub architecture. The central component is an XPUB/XSUB broker that routes messages.

Architecture



XPUB/XSUB Broker

The broker is the central message hub:

- **XSUB** (tcp://*:7500) — Publishers connect here
- **XPUB** (tcp://*:7501) — Subscribers connect here

The broker forwards messages in both directions:

- Data: XSUB → XPUB
- Subscriptions: XPUB → XSUB

Starting the Broker

```
snapper broker
```

Or programmatically:

```
from snapper.messaging.infrastructure.broker import ZmqBrokerThread

broker = ZmqBrokerThread()
broker.start()
# ...
broker.stop()
```

Topics

Topic format varies by category (see per-category tables below).

Market Data

Topic	Description
market.kraken.BTC-USD.candles.1h	Hourly BTC/USD candles from Kraken
market.kraken.BTC-USD.ticks	BTC/USD ticks from Kraken
market.polygon.AAPL.candles.1d	Daily AAPL candles from Polygon

Signals

Topic	Description
signals.paper.BTC-USD.rsi_btc_1h	Paper trading signals
signals.kraken.BTC-USD.live	Live signals from Kraken

Orders & Executions

Topic	Description
orders.commands.kraken.BTC-USD.submit	Order requests for Kraken
orders.events.kraken.BTC-USD.executed	Order executions from Kraken
orders.events.kraken.BTC-USD.submitted	Order submitted events

Topic	Description
orders.events.kraken.BTC-USD.accepted	Order accepted events
orders.events.kraken.BTC-USD.rejected	Order rejected events
orders.events.kraken.BTC-USD.cancelled	Order cancelled events
orders.events.kraken.BTC-USD.expired	Order expired events
orders.events.kraken.BTC-USD.replaced	Order replaced events

TradeCommand, VenueEvent, and TradeProjectionCheckpoint are durable database artifacts used by the trade runtime. They are not additional ZMQ topics.

System

Topic	Description
system.heartbeats.executor.{exchange}	Executor heartbeat (e.g. executor.kraken)
system.heartbeats.strategy.{name}	Strategy heartbeat (e.g. strategy.rsi_btc_1h)
system.heartbeats.feed.{exchange}	Feed heartbeat (e.g. feed.kraken, feed.paper.kraken)
system.settings	Configuration change notifications
system.symbol_aliases	Symbol cache invalidation
system.replay.start	Data replay start
system.replay.end	Replay end

Heartbeat component values use dot notation matching the topic path after system.heartbeats.: executor.kraken, strategy.rsi_btc_1h, feed.kraken, feed.paper.kraken.

Message Data Classes

All messages use typed Data classes (Pydantic) from `snapper.messaging.schemas.data`. Every Data class inherits from `StrictDataSchema` and carries:

Field	Type	Description
public_id	string	UUID7 external identifier, stable across REST and WS
type	string	Literal type discriminator
timestamp	datetime	Message creation time
session_id	string	UUID7 of the producer process session (required at construction)

Field	Type	Description
sequence_id	int	Monotonic counter per topic within the session (required at construction)

session_id and sequence_id are allocated by producers from a shared SequenceTracker before the payload is constructed. MessagePublisher.send() serializes and forwards the already-complete payload without modifying provenance. Consumers can use these fields to detect message loss and producer restarts.

The old messaging.schemas.messages module still exists but only contains parse_message() and MessageParseError.

TickData

```
from snapper.messaging.schemas.data import TickData

tick = TickData(
    instrument="BTC-USD",
    exchange="kraken",
    bid=41990.0,
    ask=42010.0,
    last=42000.0,
    volume=0.5,
    timestamp=datetime.now(UTC),
)
```

Fields:

Field	Type	Description
type	string	"tick"
public_id	string	UUID7 external identifier
instrument	string	Instrument symbol
exchange	string	Exchange name
bid	float None	Best bid price
ask	float None	Best ask price
last	float None	Last traded price
volume	float	Volume
timestamp	datetime	Timestamp
session_id	string	Producer session (inherited)
sequence_id	int	Monotonic counter per topic or logical stream (inherited)

CandleData

```
from snapper.messaging.schemas.data import CandleData

candle = CandleData(
    instrument="BTC-USD",
    exchange="kraken",
    timeframe="1h",
    open_at=datetime(2026, 1, 18, 12, 0, tzinfo=UTC),
    open=42000.0,
    high=42500.0,
    low=41800.0,
    close=42300.0,
    volume=1234.56,
    timestamp=datetime.now(UTC),
)
```

Fields:

Field	Type	Description
type	string	"candle"
public_id	string	UUID7 external identifier
instrument	string	Symbol
exchange	string	Exchange
timeframe	string	Timeframe (1m, 5m, 1h, etc.)
open_at	datetime	Exchange-provided candle interval start time
open	float	Open price
high	float	High
low	float	Low
close	float	Close price
volume	float	Volume
vwap	float null	Volume-weighted average price (optional)
trades	int null	Number of trades in the candle (optional)
timestamp	datetime	Timestamp

SignalData

```
from snapper.messaging.schemas.data import SignalData

signal = SignalData(
    instrument="BTC-USD",
    exchange="paper",
    side="buy",
    strength=0.85,
    reason="RSI <= 30",
    strategy_name="rsi_btc_1h",
    price=42000.0,
    fired_at=datetime.now(UTC),
)
```

Fields:

Field	Type	Description
type	string	"signal"
public_id	string	UUID7 external identifier
instrument	string	Symbol
exchange	string	Target exchange
side	string	"buy" or "sell"
strength	float	Signal strength 0.0-1.0
reason	string	Reason
strategy_name	string	Strategy name (required for paper exchange signals)
price	float	Price
fired_at	datetime	Domain time when signal was generated
timestamp	datetime	System timestamp

Paper signals (exchange == "paper") require strategy_name to be set. The schema enforces this invariant at construction time so invalid paper signals cannot be created.

OrderRequestData

```
from snapper.messaging.schemas.data import OrderRequestData

order = OrderRequestData(
    instrument="BTC-USD",
    exchange="kraken",
    client_order_id="ord_123",
    side="buy",
    order_type="limit",
)
```

```
price=42000.0,  
quantity=0.1,  
strategy_id="rsi_btc_1h",  
mode="live",  
)
```

ExecutionData

```
from snapper.messaging.schemas.data import ExecutionData  
  
execution = ExecutionData(  
    instrument="BTC-USD",  
    exchange="kraken",  
    client_order_id="ord_123",  
    exchange_order_id="KRAKEN-456",  
    side="buy",  
    price=42000.0,  
    size=0.1,  
    last_size=0.1,  
    last_price=42000.0,  
    fee=0.001,  
    fee_asset="USD",  
    status="filled",  
)
```

OrderData

```
from snapper.messaging.schemas.data import OrderData  
  
order_status = OrderData(  
    instrument="BTC-USD",  
    exchange="kraken",  
    client_order_id="ord_123",  
    exchange_order_id="KRAKEN-456",  
    side="buy",  
    order_type="limit",  
    size=0.1,  
    filled_size=0.0,  
    status="accepted",  
    price=42000.0,  
)
```

HeartbeatData

```
from snapper.messaging.schemas.data import HeartbeatData  
  
heartbeat = HeartbeatData(  
    session_id="019e1a2b-0000-7000-8000-000000000001",  
    sequence_id=1,  
    component="zmq_broker",  
    status="healthy",  
)
```

```
sequence=1,  
lag_ms=5,  
)
```

Publisher

MessagePublisher (recommended)

MessagePublisher sends fully constructed payloads to an explicit stream_key. Producers allocate session_id and sequence_id from a shared SequenceTracker before constructing the payload, then pass both the topic and payload to MessagePublisher.send().

```
import zmq.asyncio  
from snapper.messaging.infrastructure.validated_socket import  
ValidatedPublisher, apply_hwm, HWM_MARKET_DATA  
from snapper.messaging.infrastructure.publisher import MessagePublisher,  
SequenceTracker  
from snapper.messaging.topics.builders import topic_for_message  
  
ctx = zmq.asyncio.Context()  
raw_socket = ctx.socket(zmq.PUB)  
apply_hwm(raw_socket, sndhwm=HWM_MARKET_DATA)  
raw_socket.connect("tcp://127.0.0.1:7500")  
  
tracker = SequenceTracker()  
publisher = MessagePublisher(ValidatedPublisher(raw_socket), tracker)  
  
topic = topic_for_message(candle_data)  
  
await publisher.send(topic, candle_data)  
publisher.close()
```

For brevity, the example assumes candle_data was already constructed with session_id and sequence_id allocated from tracker.

One SequenceTracker per component process; all MessagePublisher instances within that component share it. Counters survive socket reconnects — only a full component restart creates a new session.

Per-Topic Counters

SequenceTracker maintains monotonic counters keyed by **ZMQ topic** (e.g. market.kraken.BTC-USD.candles.1m, orders.events.kraken.BTC-USD.executed). Each topic has its own independent counter. Producers allocate provenance using the final ZMQ topic string and then pass that same topic to MessagePublisher.send().

For DB-only writes that do not flow over ZMQ (instruments, users, settings seed data), the counter key is the destination table name as a logical identifier.

For WS control and REST middleware paths, logical channel names are used:
server.control, server.telemetry, rest.control, rest.telemetry.

For paper market data or other cases where the topic cannot be derived from the payload alone, pass an explicit topic override:

```
topic = "market.paper.kraken.BTC-USD.ticks"
await publisher.send(topic, tick_data)
```

ValidatedPublisher (low-level)

Direct access to the socket wrapper without StrictDataSchema serialization helpers:

```
import zmq.asyncio
from snapper.messaging.infrastructure.validated_socket import
ValidatedPublisher, apply_hwm, HWM_MARKET_DATA

ctx = zmq.asyncio.Context()
raw_socket = ctx.socket(zmq.PUB)
apply_hwm(raw_socket, sndhwm=HWM_MARKET_DATA)
raw_socket.connect("tcp://127.0.0.1:7500")
publisher = ValidatedPublisher(raw_socket)

topic = "market.kraken.BTC-USD.candles.1h"
payload = candle_data.to_json().encode()
await publisher.send_multipart(topic, payload)
publisher.close()
```

Subscriber

Subscribing to messages via validated socket wrapper around a raw ZMQ SUB socket:

```
import zmq.asyncio
from snapper.messaging.infrastructure.validated_socket import
ValidatedSubscriber, apply_hwm, HWM_MARKET_DATA
from snapper.messaging.schemas.messages import parse_message

ctx = zmq.asyncio.Context()
raw_socket = ctx.socket(zmq.SUB)
apply_hwm(raw_socket, rcv_hwm=HWM_MARKET_DATA)
raw_socket.connect("tcp://127.0.0.1:7501") # broker XPUB
subscriber = ValidatedSubscriber(raw_socket)
subscriber.subscribe("market.kraken.BTC-USD.")

while True:
    topic, payload = await subscriber.recv_multipart()
    envelope = parse_message(payload.decode())
    print(f"Received {envelope.type} on {topic}")
```

Market Data Publisher

Built-in publisher for exchange data:

```
snapper feed --symbols "BTC-USD,ETH-USD"
```

Programmatically:

```
from snapper.messaging.publishers.kraken import KrakenMarketDataPublisher

async def run_feed():
    publisher = KrakenMarketDataPublisher(symbols=["BTC-USD", "ETH-USD"])
    await publisher.start()
```

Candle Identity Cache

On startup the publisher loads the latest candle `public_id` per (instrument, timeframe) pair from the database into an in-memory cache. When a new tick arrives for an existing candle interval (`open_at` matches), the cached `public_id` is reused so the same logical candle keeps a stable identity across ZMQ, WebSocket, and the database. When `open_at` advances to a new interval, a fresh UUID7 is minted and the cache entry is replaced. This avoids any DB reads on the hot path.

Micro-Batch DB Persistence

DB writes are decoupled from the ZMQ publish path via per-loop micro-batching. Each producer loop (candles, ticks, trades) accumulates rows in a local batch and flushes to the database on a size or age threshold:

- **Size trigger:** candles flush at 100 rows, ticks/trades at 500 rows
- **Age trigger:** all streams flush after 50 ms since the first batch item
- **Publish-first:** ZMQ delivery happens before DB persistence; subscribers receive data without waiting for the database

Thresholds are configurable via `write_buffer_flush_ms`, `write_buffer_candle_max_rows`, `write_buffer_tick_max_rows`, and `write_buffer_trade_max_rows` settings (cached at publisher start, require process restart to change).

On shutdown (launcher cancellation or direct stop), each loop flushes its remaining batch in a finally block. The `asyncio.wait` age trigger uses a persistent future that is never cancelled on timeout, preserving the underlying exchange subscription iterator.

Candle flushes that hit an `IntegrityError` (e.g. out-of-order timestamps) fall back to row-by-row retry, isolating the bad row without losing the rest of the batch. The repository `upsert_candles()` contract is unchanged.

Per-stream flush error counters (`_flush_errors`) drive the heartbeat status field: "warning" if any stream has errors, "healthy" otherwise. Counters reset to zero on the next successful flush of that same stream.

Order Executor

Service executing orders:

```
snapper executor -e kraken
```

Executor:

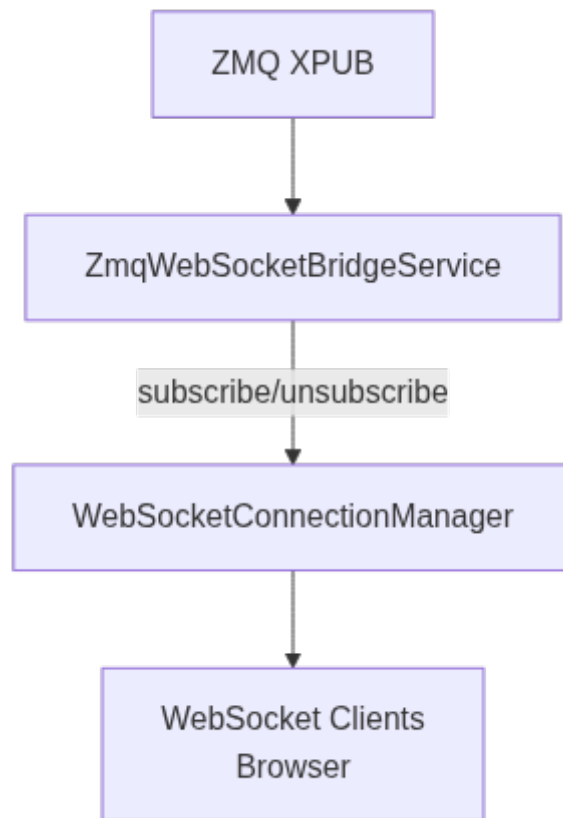
1. Subscribes to `orders.commands.{exchange}.*` topics
2. Receives `OrderRequestData` from the trade runtime, either from the direct dual-write path or the durable outbox dispatcher
3. Executes order via exchange API
4. Persists `VenueEvent` rows for accepted, fill, and terminal observations
5. Publishes `ExecutionData` or `OrderData`

In durable mode, failed `VenueEvent` persistence is treated as fail-closed on the accepted/fill paths and aborts downstream publish from the executor.

The trade runtime subscribes to `orders.events.*` to keep `TradeService` and `BalanceService` in sync during normal operation. `VenueEvent` rows are used as the durable recovery and reconciliation backbone.

ZMQ-WebSocket Bridge

Bridge between ZMQ and WebSocket for frontend:



Bridge automatically:

- Subscribes to ZMQ topics when WebSocket client subscribes
- Validates each inbound ZMQ message as JSON and runs `GapDetector.check()` before forwarding
- Drops messages with malformed JSON (logged as WARNING; counted in `invalid_messages` per topic)
- Forwards messages via `_forward_to_clients` with per-subscription backpressure
- Throttles market data per subscriber (configurable `throttle_ms`)
- Drops market data when a client exceeds `MAX_PENDING_MESSAGES_MARKET` (100)
- Disconnects slow clients on trade topics when exceeding `MAX_PENDING_MESSAGES_TRADE` (1000)
- Unsubscribes when last client disconnects
- Records control events (subscribe errors, client disconnects) to the control table

Gap Detection

`GapDetector` tracks `session_id` and `sequence_id` per received ZMQ topic and emits log messages when it finds sequence gaps or producer session resets.

```
from snapper.messaging.infrastructure.gap_detector import GapDetector

detector = GapDetector()
detector.check(received_topic, session_id, sequence_id)
```

Rules:

- Messages without provenance (`session_id == ""` or `sequence_id == 0`) are **rejected** with a WARNING log and a `rejected_unstamped` counter increment. This makes unstamped traffic observable rather than silently passing through.
- First message on a topic: sets baseline. If `sequence_id > 1` the subscriber joined mid-stream (logged INFO).
- Same topic, new `session_id`: producer restarted (logged INFO, counter resets).
- In-order message (`sequence_id == expected`): accepted silently.
- Gap (`sequence_id > expected`): logs WARNING with the missing range, then advances.
- Duplicate / reorder (`sequence_id < expected`): logs DEBUG, state unchanged.

`GapDetector.check()` returns `True` when the message carries valid provenance and was processed, `False` when rejected as unstamped.

`GapDetector` is wired into the ZMQ-WebSocket bridge and into subscriber loops inside the executor, trade runtime coordinator, and `strategy_listen_loop()`.

Message Logger

Debug tool for monitoring ZMQ traffic:

```
snapper zmq-logger --payload --audit-file logs/zmq.jsonl
```

Logs all messages passing through the broker.

Configuration

Endpoints in `.env`:

```
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501
```

Throttling

`TopicSchema` defines throttling per topic:

```
TopicSchema(  
    name="market",  
    pattern="market.",  
    throttle_ms=100, # Max 10 msg/s  
    ...  
)
```

Message Validation

ValidatedPublisher and ValidatedSubscriber validate **topic strings**, not message payloads. ValidatedPublisher.send_multipart() calls validate_topic() before sending; ValidatedSubscriber.subscribe() calls validate_subscription_pattern() before subscribing. Both raise TopicValidationError on invalid topics.

High Water Mark (HWM) Policy

ZMQ sockets use explicit high water marks via apply_hwm() from validated_socket.py to bound queue depth and prevent silent message loss:

Tier	Constant	Value	Used by
Order flow	HWM_ORDER_FLOW	0 (unlimited)	Executors, trade runtime coordinator
Broker	HWM_BROKER	10 000	XSUB (rcvhwm), XPUB (sndhwm)
Market data	HWM_MARKET_DATA	5 000	Publishers, strategies, bridge, settings, symbol updaters
Audit	HWM_AUDIT	10 000	Message logger

apply_hwm() must be called **before** connect() or bind().

Retry and Resilience

Sockets automatically:

- Reconnect on connection loss
- Buffer messages when broker unavailable
- LINGER=0 on close (discard pending)
- HWM applied before connect/bind (see above)

Topic Derivation

`topic_for_message()` in `snapper.messaging.topics.builders` derives the canonical ZMQ topic from a typed `Data` class. Producers typically call it before `MessagePublisher.send()`.

```
from snapper.messaging.topics.builders import topic_for_message

topic = topic_for_message(candle_data)
await publisher.send(topic, candle_data)
```

Paper market data topics include a `source_exchange` segment that is not carried in the payload, so they must always be published with an explicit topic override.

WebSocket Message Provenance

All WebSocket protocol messages (authentication, subscription management, ping/pong, errors) inherit directly from `StrictDataSchema`, so they carry the same provenance fields as ZMQ data payloads: `public_id`, `session_id`, `sequence_id`, and `timestamp`. Every payload item in the system — whether it flows over ZMQ, REST, or WebSocket — has a uniform provenance envelope.

Audit Tables: Control and Telemetry

Two destination tables provide always-available observability for non-domain traffic:

- **control** — Always-on audit for commands, authentication events, subscribe/unsubscribe messages, and REST mutation requests. Recording is wired in three places:
- **WS handlers** — `_record_ws_control()` records auth, subscribe, unsubscribe, and error events with `transport="ws"`. Client causation linkage is extracted from inbound messages via `_extract_client_provenance()` and stored as `client_session_id` and `client_public_id` on the control row.
- **REST middleware** — `ClientProvenanceMiddleware._record_control()` records every mutation (POST) with `transport="rest"`, redacted payload, outcome (ok/error/exception), and server-side provenance.
- **ZMQ bridge** — `_record_bridge_control()` records subscribe errors and client disconnects with `transport="zmq"`.

All control writes use a finally block so the record is persisted regardless of whether the request succeeded or failed. The write is non-blocking: any DB failure is logged and swallowed so the response already sent to the client is never invalidated.

- **telemetry** — Toggleable high-volume table for pings, heartbeats, pongs, and GET read requests. Recording is gated by the `TELEMETRY_RECORDING_ENABLED` environment variable (default false). When disabled, `SequenceTracker` counters still increment — only the DB write is skipped. Recording is wired in:
- **WS handlers** — `_record_ws_telemetry()` records ping/pong/heartbeat events
- **REST middleware** — `_record_telemetry()` records GET reads (health, status, entity endpoints)

The non-blocking audit invariant applies to both tables: audit writes never reject, delay, or invalidate the primary request/message flow.

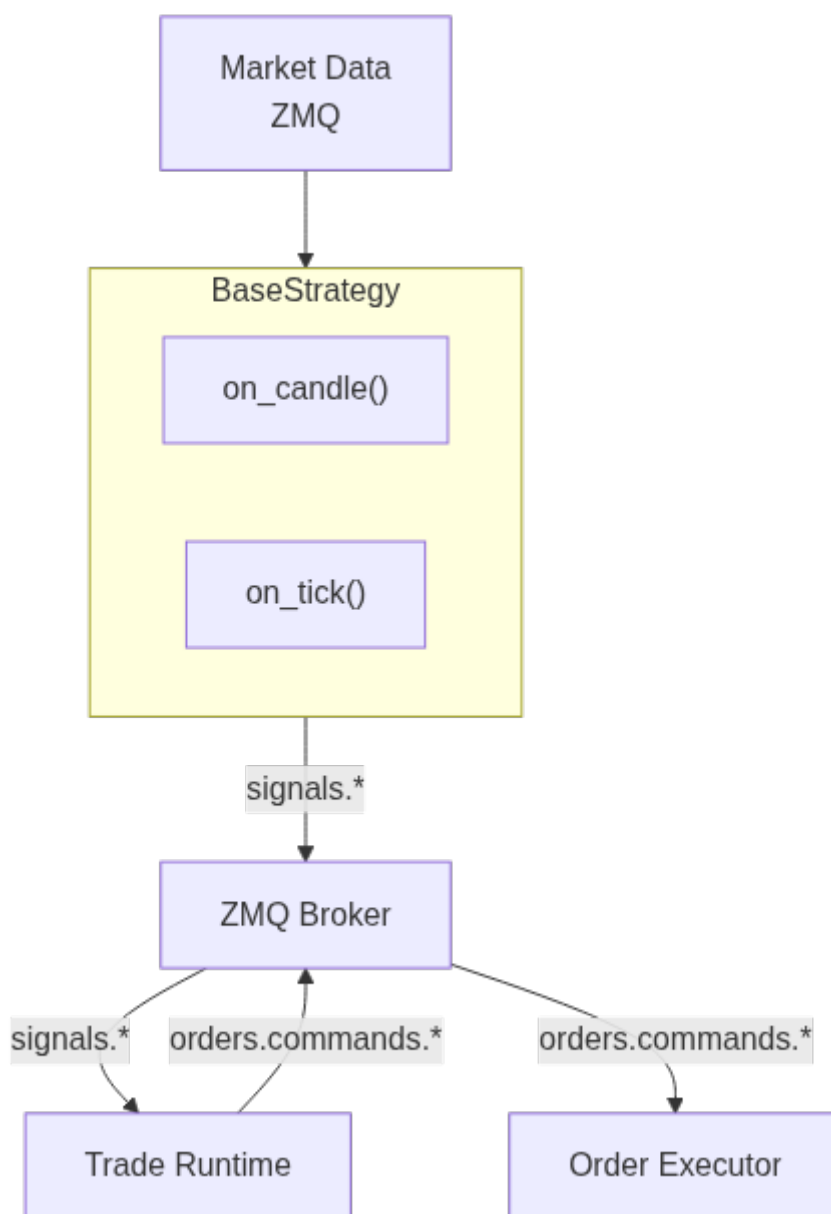
Best Practices

1. **One broker per system** — All components connect to the same broker
2. **Topic hierarchy** — Use hierarchy for filtering (`market.kraken.*`)
3. **Data types** — Always use typed Data classes from `messaging.schemas.data`
4. **Provenance** — Use `MessagePublisher` (not `ValidatedPublisher` directly) so every message carries `session_id` and `sequence_id` for gap detection
5. **One SequenceTracker per component** — Create it once at `start()` and share across all publisher instances; restart creates a new session
6. **Heartbeats** — Keep component-specific heartbeat cadences small and regular
7. **Graceful shutdown** — Close sockets with `LINGER=0`

Trading Strategies

Snapper provides a framework for creating trading strategies based on ZeroMQ messaging. Strategies subscribe to market data and publish signals. Strategies produce intent only; the trade runtime owns order lifecycle, positions, and balance projections.

Strategy Architecture



Creating Strategies

Basic Structure

```
from snapper.strategies.base import BaseStrategy, StrategySignal, StrategyConfig
from snapper.strategies.decorators import register_strategy,
create_strategy_process
from snapper.messaging.schemas.data import CandleData

@register_strategy("MyStrategy")
@create_strategy_process(
    process_name="my_strategy_btc",
    default_config={
        "name": "my_strategy_btc",
        "inputs": ["market.kraken.BTC-USD.candles.1h"],
        "outputs": ["BTC-USD"],
        "exchange": "paper",
        "params": {
            "threshold": 0.5,
            "period": 14,
        },
    },
)
class MyStrategy(BaseStrategy):
    """Strategy description in docstring."""

    def __init__(self, config: StrategyConfig) -> None:
        """Initialize strategy."""
        super().__init__(config)
        self.threshold = self.params.get("threshold", 0.5)
        self.period = self.params.get("period", 14)

    async def on_candle(self, instrument: str, candle: CandleData) ->
StrategySignal | None:
        """Process candle and generate signal."""
        candles = self.candle_buffer.get(instrument, [])
        if len(candles) < self.period:
            return None

        closes = [c.close for c in candles]
        current_price = closes[-1]

        if self._should_buy(closes):
            return StrategySignal(
                instrument=instrument,
                side="buy",
                strength=1.0,
                price=current_price,
                reason="Buy condition met",
            )

        if self._should_sell(closes):
            return StrategySignal(
                instrument=instrument,
                side="sell",
                strength=1.0,
                price=current_price,
                reason="Sell condition met",
            )
```

```

        return None

    def _should_buy(self, closes: list[float]) -> bool:
        """Buy condition logic."""
        return False

    def _should_sell(self, closes: list[float]) -> bool:
        """Sell condition logic."""
        return False

    async def reset(self) -> None:
        """Reset strategy state (for replay)."""
        pass

```

Decorators

@register_strategy(name)

Registers strategy class in factory under given name.

```

@register_strategy("RSIReversion")
class RSIReversion(BaseStrategy):
    ...

```

@create_strategy_process(process_name, default_config)

Creates process wrapper for strategy and registers in process manager.

```

@create_strategy_process(
    process_name="strategy_rsi_eth",
    default_config={
        "name": "rsi_eth_1h",
        "inputs": ["market.kraken.ETH-USD.candles.1h"],
        "outputs": ["ETH-USD"],
        "exchange": "paper",
        "params": {"period": 14, "upper": 70, "lower": 30},
    },
)
class RSIReversion(BaseStrategy):
    ...

```

Strategy Configuration

StrategyConfig

Field	Type	Description
name	string	Unique strategy instance name
strategy_class	string	Strategy class name
inputs	list[str]	List of ZMQ topics to subscribe
outputs	list[str]	List of instruments for signals
exchange	string	Target exchange (paper, kraken, zonda, walutomat)
params	dict	Strategy-specific parameters

Input Topics

Common formats:

- `market.{exchange}.{instrument}.candles.{timeframe}` — Candle streams
- `market.{exchange}.{instrument}.ticks` — Tick streams
- `market.{exchange}.{instrument}.trades` — Trade streams

Examples:

- `market.kraken.BTC-USD.candles.1h` — Hourly BTC/USD candles from Kraken
- `market.kraken.ETH-USD.ticks` — ETH/USD ticks from Kraken
- `market.polygon.AAPL.trades` — AAPL trade tape from Polygon

Output Topics (Signals)

Generated automatically:

- Paper: `signals.paper.{instrument}.{strategy_name}`
- Live: `signals.{exchange}.{instrument}.live`

StrategySignal Structure

```
@dataclass
class StrategySignal:
    """Trading signal."""

    instrument: str
    side: TradeSide
    strength: float
```

```
reason: str
price: float
timestamp: datetime | None
```

Built-in Strategies

RSIReversion

Mean-reversion strategy based on RSI.

Parameters:

Parameter	Default	Description
period	14	RSI period
upper	70.0	Overbought threshold (sell)
lower	30.0	Oversold threshold (buy)
cooldown	2	Candles between signals

Logic:

- Buy when RSI $\leq$ lower
- Sell when RSI $\geq$ upper

```
from snapper.strategies.rsi import RSIReversion
```

MACDCrossover

Trend-following strategy based on MACD.

Parameters:

Parameter	Default	Description
fast	12	Fast EMA
slow	26	Slow EMA
signal_period	9	StrategySignal line period

Logic:

- Buy when MACD histogram crosses from negative to positive
- Sell when histogram crosses from positive to negative

```
from snapper.strategies.macd import MACDCrossover
```

Cointegration

Pairs trading strategy based on cointegration.

```
from snapper.strategies.cointegration import CointegrationPairs
```

Data Access

Candle Buffer

```
async def on_candle(self, instrument: str, candle: CandleData) ->
StrategySignal | None:
    candles = self.candle_buffer.get(instrument, [])

    recent = candles[-20:]

    import pandas as pd
    closes = pd.Series([c.close for c in candles])
```

CandleData

```
from snapper.messaging.schemas.data import CandleData
```

Fields:

Field	Type	Description
type	string	"candle"
public_id	string	UUID7 external identifier
open	float	Open price
high	float	High
low	float	Low
close	float	Close price
volume	float	Volume
timestamp	datetime	Timestamp
timeframe	string	Timeframe

Field	Type	Description
instrument	string	Symbol
exchange	string	Exchange

Technical Indicators

RSI

```
from snapper.indicators.rsi import rsi
import pandas as pd

closes = pd.Series([c.close for c in candles])
rsi_values = rsi(closes, period=14)
current_rsi = rsi_values.iloc[-1]
```

MACD

```
from snapper.indicators.ta_lib_adapter import macd

macd_line, signal_line, histogram = macd(closes, fast=12, slow=26, signal=9)
```

TA-Lib (via adapter)

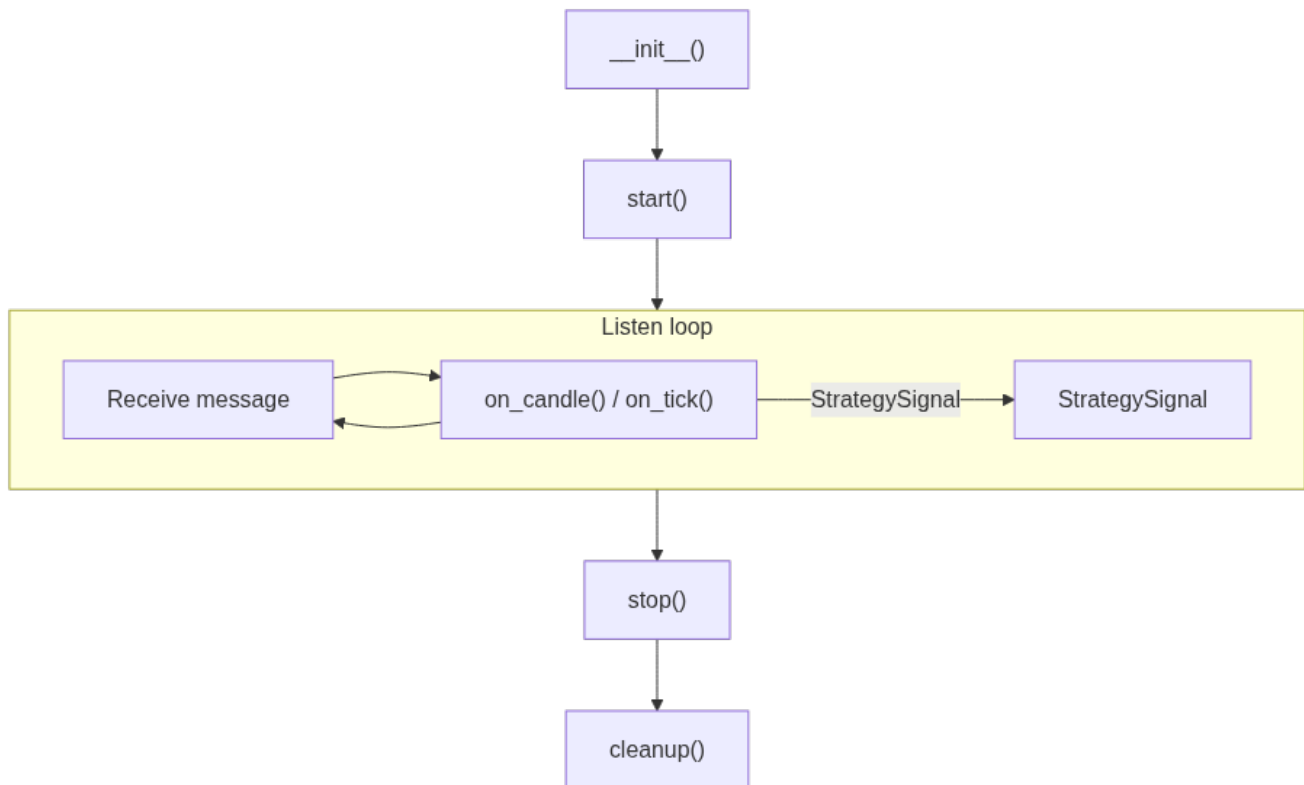
```
from snapper.indicators.ta_lib_adapter import rsi, macd
```

Configuration Validation

Framework automatically validates:

1. **Strategy name** — cannot be empty
2. **Inputs** — at least one required
3. **Outputs** — at least one instrument must be defined
4. **Exchange** — must be one of: paper, kraken, zonda, walutomat
5. **Instruments** — must be available on selected exchange
6. **Paper/live mixing** — mixing paper inputs with live exchange not allowed

Strategy Lifecycle



Running Strategies

Via Process Manager (recommended)

Strategies are automatically managed by the server:

```
snapper server
```

Strategies can be enabled/disabled in the dashboard.

Standalone (for testing)

```
import asyncio
from snapper.strategies.factory import StrategyFactory
from snapper.strategies.base import StrategyConfig

config = StrategyConfig(
    name="test_rsi",
    strategy_class="RSIReversion",
    inputs=["market.kraken.BTC-USD.candles.1h"],
    outputs=["BTC-USD"],
    exchange="paper",
    params={"period": 14, "upper": 70, "lower": 30},
)
```

```

factory = StrategyFactory()
strategy = factory.create_strategy(config)

async def main():
    await strategy.start()

asyncio.run(main())

```

Testing Strategies

Unit Tests

```

import pytest
from snapper.strategies.rsi import RSIRversion
from snapper.strategies.base import StrategyConfig
from snapper.messaging.schemas.data import CandleData

@pytest.fixture
def rsi_strategy() -> RSIRversion:
    config = StrategyConfig(
        name="test_rsi",
        strategy_class="RSIRversion",
        inputs=["market.kraken.BTC-USD.candles.1h"],
        outputs=["BTC-USD"],
        exchange="paper",
        params={"period": 14, "upper": 70, "lower": 30},
    )
    return RSIRversion(config)

async def test_buy_signal_on_low_rsi(rsi_strategy: RSIRversion) -> None:
    # Prepare data with low RSI
    ...
    signal = await rsi_strategy.on_candle("BTC-USD", candle)
    assert signal is not None
    assert signal.side == "buy"

```

Backtesting

Use replay data from paper topics:

```

default_config={
    "inputs": ["replay.BTC-USD.candles.1h"],
    "exchange": "paper",
    ...
}

```